



UT 6. Seguridad informática y comunicaciones seguras

Sistemas informáticos

DAM-DAW

Curso académico 2025/2026 - Edición Online - Rev. 2025093003

Tabla de contenidos

Tabla de contenidos	2
Ficha de la Unidad de Trabajo	6
Índice de la unidad de trabajo	7
UT6 – Seguridad informática y comunicaciones seguras	9
6.1. Cuentas de usuario y contraseñas seguras	9
6.1. Cuentas de usuario y contraseñas seguras	12
6.2. Permisos de acceso y directivas locales	22
6.3 Control de procesos y servicios del sistema.....	29
6.3.1 Conceptos fundamentales de procesos y servicios	30
6.3.1.1 Procesos.....	30
6.3.1.2 Hilos (Threads)	30
6.3.1.3 Servicios y demonios	31
6.3.2 Gestión de procesos en Windows.....	31
6.3.2.1 Introducción	31
6.3.2.2 Herramientas gráficas	31
6.3.2.3 Comandos y PowerShell	32
6.3.3 Gestión de servicios en Windows	33
6.3.3.1 Introducción	33
6.3.3.2 Comandos PowerShell para servicios	33
6.3.3.3 Ejemplo: Script para reiniciar un servicio si está detenido	34
6.3.4 Gestión de procesos en Linux.....	34
6.3.4.1 Introducción	34
6.3.4.2 Herramientas de monitorización.....	34
6.3.4.3 Comandos básicos	35
6.3.4.4 Monitorización avanzada	35
6.3.5 Gestión de servicios en Linux (systemd).....	36
6.3.5.1 Introducción	36
6.3.5.2 Comandos básicos	36
6.3.6 Monitorización y control avanzado	37
6.3.6.1 Renice y prioridad de procesos.....	37
6.3.6.2 Scripts para monitorizar procesos y reiniciar servicios	37

6.3.7 Buenas prácticas para control de procesos y servicios	38
6.4 Herramientas de optimización y diagnóstico	39
6.4 Introducción	39
6.4.1 Herramientas de optimización en Windows.....	39
6.4.1.1 Administrador de tareas (Task Manager)	39
6.4.1.2 Monitor de recursos (Resource Monitor)	40
6.4.1.3 Liberador de espacio en disco (Disk Cleanup)	40
6.4.1.4 Desfragmentador de disco	40
6.4.1.5 Comandos para optimización y diagnóstico	41
6.4.1.6 Ejemplo: Uso de PowerShell para monitorizar procesos y recursos	41
6.4.2 Herramientas de optimización en Linux	41
6.4.2.1 Monitorización de recursos	41
6.4.2.2 Análisis de disco y sistema de archivos	42
6.4.2.3 Diagnóstico de red	42
6.4.2.4 Análisis de logs	42
6.4.2.5 Ejemplo: Script para monitorizar uso de CPU y enviar alerta	43
6.4.3 Herramientas de diagnóstico avanzadas.....	43
6.4.3.1 Windows Performance Toolkit (WPT)	43
6.4.3.2 Sysinternals Suite	43
6.4.3.3 Herramientas de diagnóstico en Linux	44
6.4.4 Buenas prácticas para optimización y diagnóstico	44
6.5 Comunicaciones seguras: VPN, HTTPS, SSH.....	45
6.5 Introducción	45
6.5.1 VPN (Redes Privadas Virtuales)	45
6.5.1.1 Concepto y funcionamiento.....	45
6.5.1.2 Protocolos comunes.....	46
6.5.1.3 Ejemplo de configuración básica OpenVPN en Linux.....	46
6.5.2 HTTPS (HTTP Secure)	46
6.5.2.1 Concepto y funcionamiento.....	46
6.5.2.2 Certificados digitales	47
6.5.2.3 Configuración básica en Apache (Linux)	47
6.5.3 SSH (Secure Shell).....	47
6.5.3.1 Concepto y funcionamiento.....	47
6.5.3.2 Autenticación	48
6.5.3.3 Comandos básicos y opciones.....	48

6.5.4 Buenas prácticas para comunicaciones seguras.....	49
6.6 Configuración de firewall (ufw, Windows Defender Firewall)	49
6.6 Introducción	49
6.6.1 Windows Defender Firewall.....	50
6.6.1.1 Conceptos básicos.....	50
6.6.1.2 Configuración mediante interfaz gráfica	50
6.6.1.3 Configuración mediante PowerShell	50
6.6.1.4 Opciones importantes explicadas.....	51
6.6.1.5 Ejemplo avanzado: Permitir RDP solo desde una IP específica	52
6.6.2 UFW (Uncomplicated Firewall) en Linux	52
6.6.2.1 Conceptos básicos.....	52
6.6.2.2 Comandos básicos	52
6.6.2.3 Opciones avanzadas explicadas	53
6.6.2.4 Ejemplo avanzado: Configurar UFW para servidor web seguro	54
6.6.3 Buenas prácticas para configuración de firewall.....	54
6.7 Comprobación del estado de red y puertos.....	55
6.7 Introducción	55
6.7.1 Herramientas para comprobar puertos abiertos	55
6.7.1.1 netstat.....	55
6.7.1.2 ss (socket statistics).....	56
6.7.2.2 netcat (nc).....	58
6.7.3 Comprobación de conectividad y diagnóstico	58
6.7.3.1 ping.....	58
6.7.3.2 traceroute / tracert.....	59
6.7.4 Buenas prácticas para comprobación de red y puertos.....	59
6.8 Monitorización y alertas	60
6.8 Introducción	60
6.8.1 Herramientas de monitorización en Windows	60
6.8.1.1 Monitor de rendimiento (Performance Monitor)	60
6.8.1.2 PowerShell para monitorización.....	61
6.8.2 Herramientas de monitorización en Linux	61
6.8.2.1 Nagios.....	61
6.8.2.2 Zabbix	62
6.8.2.3 Comandos básicos para monitorización rápida	62
6.8.2.4 Ejemplo de script para alertar uso alto de CPU.....	62

6.8.3 Configuración de alertas efectivas	63
6.8.4 Buenas prácticas en monitorización y alertas	63
¿Qué has aprendido?	66
Fuentes de información	67
Glosario	67
Actividades de evaluación	68
Tabla de relación entre contenidos y criterios de evaluación (CE)	72
Rúbrica de evaluación	73

Ficha de la Unidad de Trabajo

Unidad de Trabajo 6: Seguridad informática y comunicaciones seguras

Carga lectiva estimada: 20 horas

Resultados de Aprendizaje trabajados:

- **RA4. Administra el sistema operativo aplicando criterios de seguridad y eficacia.**
- **RA6. Opera sistemas en red gestionando sus recursos e identificando las restricciones de seguridad existentes.**
- **RA7. Elabora documentación técnica y administra aplicaciones de propósito general.**

Criterios de Evaluación trabajados:

- **RA4CEa, RA4CEb, RA4CEc, RA4CEd**
- **RA6CEa, RA6CEb, RA6CEc, RA6CEd, RA6CEe, RA6CEf**
- **RA7CEa, RA7CEc**

Índice de la unidad de trabajo

1. Introducción a la seguridad informática y comunicaciones seguras
 - 1.1. Importancia de la protección de la información
 - 1.2. Principios de confidencialidad, integridad y disponibilidad
2. Cuentas de usuario y contraseñas seguras
 - 2.1. Tipos de usuarios en Windows
 - 2.2. Tipos de usuarios en Linux
 - 2.3. Políticas de contraseñas seguras
 - 2.4. Comandos avanzados para la gestión de usuarios
 - 2.5. Auditoría de cuentas y contraseñas
3. Permisos de acceso y directivas locales
 - 3.1. Modelos de control de acceso: DAC, MAC, RBAC
 - 3.2. Permisos en Windows (NTFS, icaccls, gpedit.msc)
 - 3.3. Permisos en Linux (chmod, chown, setfacl)
 - 3.4. Directivas locales y de grupo
 - 3.5. Ejemplos prácticos comparativos
4. Control de procesos y servicios del sistema
 - 4.1. Conceptos de procesos, hilos y servicios
 - 4.2. Herramientas de gestión en Windows (Task Manager, PowerShell)
 - 4.3. Herramientas de gestión en Linux (top, ps, systemctl)
 - 4.4. Monitorización avanzada y scripts
 - 4.5. Buenas prácticas en la gestión de servicios
5. Herramientas de optimización y diagnóstico
 - 5.1. Herramientas en Windows (Task Manager, Perfmon, Sysinternals)
 - 5.2. Herramientas en Linux (top, htop, vmstat, iostat, journalctl)
 - 5.3. Análisis de logs y diagnósticos avanzados
 - 5.4. Buenas prácticas de optimización
6. Comunicaciones seguras: VPN, HTTPS y SSH
 - 6.1. Concepto de VPN y protocolos comunes (IPSec, OpenVPN, WireGuard)
 - 6.2. HTTPS: certificados digitales y configuración
 - 6.3. SSH: comandos básicos, autenticación por claves y SCP
 - 6.4. Buenas prácticas en comunicaciones seguras
7. Configuración de firewalls
 - 7.1. Windows Defender Firewall (GUI y PowerShell)
 - 7.2. UFW en Linux (comandos básicos y avanzados)
 - 7.3. Buenas prácticas en la gestión de reglas
8. Comprobación del estado de red y puertos
 - 8.1. Herramientas básicas (ping, traceroute, netstat, ss)

8.2. Herramientas de escaneo (nmap, netcat)

8.3. Buenas prácticas en auditoría de red

9. Monitorización y alertas

9.1. Monitorización en Windows (Perfmon, PowerShell)

9.2. Monitorización en Linux (Nagios, Zabbix, comandos básicos)

9.3. Configuración de alertas efectivas

9.4. Buenas prácticas

UT6 – Seguridad informática y comunicaciones seguras

La seguridad informática y las comunicaciones seguras son pilares fundamentales en el ámbito de la informática, especialmente en un entorno donde la información digital es un activo crítico. La protección de los sistemas, datos y redes frente a accesos no autorizados, ataques maliciosos o fallos técnicos es esencial para garantizar la continuidad y fiabilidad de los servicios tecnológicos.

Esta unidad temática aborda los conceptos, técnicas y herramientas necesarias para proteger los sistemas informáticos y garantizar la confidencialidad, integridad y disponibilidad de los datos. Se profundiza en la gestión de cuentas de usuario, contraseñas seguras, permisos de acceso, control de procesos y servicios del sistema, así como en la configuración y uso de herramientas de optimización y diagnóstico.

Además, se estudian las comunicaciones seguras mediante tecnologías como VPN, HTTPS y SSH, y la configuración de firewalls para proteger las redes. Finalmente, se establecen criterios básicos de ciberseguridad para mantener un entorno seguro y resiliente frente a amenazas externas e internas.

En esta introducción se presenta un panorama general de la importancia de la seguridad informática, destacando la necesidad de implementar políticas y prácticas robustas para mitigar riesgos y proteger la infraestructura tecnológica. Se enfatiza la evolución constante de las amenazas y la importancia de la formación continua para los profesionales del sector.

El objetivo es proporcionar una base sólida que permita al estudiante comprender y aplicar los conceptos técnicos que se desarrollarán en los apartados siguientes, asegurando un aprendizaje integral y práctico.

6.1. Cuentas de usuario y contraseñas seguras

Introducción extensa

La gestión de cuentas de usuario es uno de los pilares fundamentales en la seguridad informática. Cada cuenta representa una identidad digital que permite a un usuario acceder a recursos y servicios dentro de un sistema operativo o red. La correcta administración de estas cuentas es vital para evitar accesos no autorizados, proteger la integridad de los datos y garantizar la trazabilidad de las acciones realizadas en el sistema.

En sistemas modernos, las cuentas de usuario no solo identifican a personas, sino también a servicios, aplicaciones y procesos que requieren permisos específicos para funcionar correctamente. Por ello, es imprescindible entender los diferentes tipos de cuentas, sus características, y cómo configurar contraseñas seguras que dificulten la acción de atacantes.

Las contraseñas son la primera línea de defensa para proteger una cuenta. Sin embargo, la simple existencia de una contraseña no garantiza seguridad. Es necesario que estas cumplan con criterios de robustez, como longitud, complejidad y periodicidad de cambio. Además, la implementación de políticas de bloqueo tras intentos fallidos y el uso de mecanismos complementarios como la autenticación multifactor (MFA) aumentan significativamente la protección.

En este apartado se profundizará en los tipos de usuarios que existen en los sistemas Windows y Linux, las características y permisos asociados a cada tipo, y se presentarán comandos avanzados para la creación, modificación y gestión de cuentas. También se abordarán las mejores prácticas para la configuración de contraseñas seguras y la administración de políticas de seguridad relacionadas.

Tipos de usuarios en sistemas Windows

En Windows, las cuentas de usuario se clasifican principalmente en función de los permisos y roles asignados. Los tipos más comunes son:

1. **Administrador**

Cuenta con privilegios completos para realizar cualquier acción en el sistema, incluyendo la instalación de software, configuración de hardware, gestión de cuentas y acceso a todos los archivos. Es la cuenta con mayor nivel de control.

2. **Usuario estándar**

Cuenta con permisos limitados para realizar tareas cotidianas como ejecutar aplicaciones y cambiar configuraciones personales, pero no puede modificar configuraciones del sistema ni instalar software que afecte a otros usuarios.

3. **Invitado**

Cuenta temporal con permisos muy restringidos, diseñada para usuarios que necesitan acceso limitado y temporal. No puede instalar software ni cambiar configuraciones.

4. **Cuentas de servicio**

Utilizadas por servicios y aplicaciones para ejecutar procesos con permisos específicos. No están destinadas a usuarios humanos.

5. **Cuentas integradas**

Windows incluye cuentas integradas como SYSTEM, LOCAL SERVICE y

NETWORK SERVICE que tienen permisos especiales para tareas del sistema operativo.

Tipos de usuarios en sistemas Linux

En Linux, la gestión de usuarios se basa en un modelo similar pero con diferencias en la nomenclatura y estructura:

1. Root

Usuario con privilegios totales, equivalente al administrador en Windows. Puede realizar cualquier acción en el sistema, incluyendo la modificación de archivos críticos y configuración del sistema.

2. Usuarios normales

Cuentas creadas para usuarios humanos con permisos limitados. Pueden ejecutar programas y modificar archivos dentro de su propio directorio home, pero no pueden realizar cambios que afecten al sistema global.

3. Usuarios del sistema

Cuentas creadas para ejecutar servicios y procesos del sistema, como www-data para servidores web o mysql para bases de datos. Estas cuentas tienen permisos específicos y no están destinadas a acceso interactivo.

4. Usuarios sin login

Cuentas configuradas para no permitir acceso interactivo, usadas para tareas específicas o servicios.

Comandos avanzados para gestión de usuarios en Windows

Windows ofrece una variedad de comandos para crear, modificar y gestionar usuarios desde la línea de comandos o PowerShell. A continuación, se detallan comandos con opciones avanzadas:

Creación de usuario con opciones

```
net user nombre_usuario contraseña /add /fullname:"Nombre Completo"  
/comment:"Descripción del usuario" /expires:never /passwordchg:no  
/passwordreq:yes
```

- /fullname:"Nombre Completo": Define el nombre completo del usuario.
- /comment:"Descripción del usuario": Añade un comentario descriptivo.
- /expires:never: La cuenta no expira.
- /passwordchg:no: El usuario no puede cambiar la contraseña.

- /passwordreq:yes: La contraseña es obligatoria.

Modificación de usuario

```
net user nombre_usuario /active:yes
```

```
net user nombre_usuario /passwordchg:yes
```

```
net user nombre_usuario /expires:01/01/2025
```

- /active:yes/no: Activa o desactiva la cuenta.
- /passwordchg:yes/no: Permite o impide que el usuario cambie la contraseña.
- /expires:fecha: Establece la fecha de expiración de la cuenta.

Asignar usuario a grupo

```
net localgroup Administradores nombre_usuario /add
```

```
net localgroup Usuarios nombre_usuario /delete
```

- Añade o elimina usuarios de grupos locales para gestionar permisos.

Ejemplo completo en PowerShell

```
New-LocalUser -Name "usuario1" -Password (ConvertTo-SecureString  
"ContraseñaSegura123!" -AsPlainText -Force) -FullName "Usuario de Prueba" -  
Description "Cuenta para pruebas" -PasswordNeverExpires $true
```

```
Add-LocalGroupMember -Group "Administradores" -Member "usuario1"
```

6.1. Cuentas de usuario y contraseñas seguras

Introducción

La gestión de cuentas de usuario es uno de los pilares fundamentales en la seguridad informática. Cada cuenta representa una identidad digital que permite a un usuario acceder a recursos y servicios dentro de un sistema operativo o red. La correcta administración de estas cuentas es vital para evitar accesos no autorizados, proteger la integridad de los datos y garantizar la trazabilidad de las acciones realizadas en el sistema.

En sistemas modernos, las cuentas de usuario no solo identifican a personas, sino también a servicios, aplicaciones y procesos que requieren permisos específicos para funcionar correctamente. Por ello, es imprescindible entender los diferentes tipos de cuentas, sus características, y cómo configurar contraseñas seguras que dificulten la acción de atacantes.

Las contraseñas son la primera línea de defensa para proteger una cuenta. Sin embargo, la simple existencia de una contraseña no garantiza seguridad. Es necesario que estas cumplan con criterios de robustez, como longitud, complejidad y periodicidad de cambio. Además, la implementación de políticas de bloqueo tras intentos fallidos y el uso de mecanismos complementarios como la autenticación multifactor (MFA) aumentan significativamente la protección.

En este apartado se profundizará en los tipos de usuarios que existen en los sistemas Windows y Linux, las características y permisos asociados a cada tipo, y se presentarán comandos avanzados para la creación, modificación y gestión de cuentas. También se abordarán las mejores prácticas para la configuración de contraseñas seguras y la administración de políticas de seguridad relacionadas.

Tipos de usuarios en sistemas Windows

En Windows, las cuentas de usuario se clasifican principalmente en función de los permisos y roles asignados. Los tipos más comunes son:

1. **Administrador**

Cuenta con privilegios completos para realizar cualquier acción en el sistema, incluyendo la instalación de software, configuración de hardware, gestión de cuentas y acceso a todos los archivos. Es la cuenta con mayor nivel de control.

2. **Usuario estándar**

Cuenta con permisos limitados para realizar tareas cotidianas como ejecutar aplicaciones y cambiar configuraciones personales, pero no puede modificar configuraciones del sistema ni instalar software que afecte a otros usuarios.

3. **Invitado**

Cuenta temporal con permisos muy restringidos, diseñada para usuarios que necesitan acceso limitado y temporal. No puede instalar software ni cambiar configuraciones.

4. **Cuentas de servicio**

Utilizadas por servicios y aplicaciones para ejecutar procesos con permisos específicos. No están destinadas a usuarios humanos.

5. **Cuentas integradas**

Windows incluye cuentas integradas como SYSTEM, LOCAL SERVICE y

NETWORK SERVICE que tienen permisos especiales para tareas del sistema operativo.

Tipos de usuarios en sistemas Linux

En Linux, la gestión de usuarios se basa en un modelo similar, pero con diferencias en la nomenclatura y estructura:

1. **Root**

Usuario con privilegios totales, equivalente al administrador en Windows. Puede realizar cualquier acción en el sistema, incluyendo la modificación de archivos críticos y configuración del sistema.

2. **Usuarios normales**

Cuentas creadas para usuarios humanos con permisos limitados. Pueden ejecutar programas y modificar archivos dentro de su propio directorio home, pero no pueden realizar cambios que afecten al sistema global.

3. **Usuarios del sistema**

Cuentas creadas para ejecutar servicios y procesos del sistema, como www-data para servidores web o mysql para bases de datos. Estas cuentas tienen permisos específicos y no están destinadas a acceso interactivo.

4. **Usuarios sin login**

Cuentas configuradas para no permitir acceso interactivo, usadas para tareas específicas o servicios.

Comandos avanzados para gestión de usuarios en Windows

Windows ofrece una variedad de comandos para crear, modificar y gestionar usuarios desde la línea de comandos o PowerShell. A continuación, se detallan comandos con opciones avanzadas:

Creación de usuario con opciones

```
net user nombre_usuario contraseña /add /fullname:"Nombre Completo"  
/comment:"Descripción del usuario" /expires:never /passwordchg:no  
/passwordreq:yes
```

- /fullname:"Nombre Completo": Define el nombre completo del usuario.
- /comment:"Descripción del usuario": Añade un comentario descriptivo.

- /expires:never: La cuenta no expira.
- /passwordchg:no: El usuario no puede cambiar la contraseña.
- /passwordreq:yes: La contraseña es obligatoria.

Modificación de usuario

```
net user nombre_usuario /active:yes
```

```
net user nombre_usuario /passwordchg:yes
```

```
net user nombre_usuario /expires:01/01/2025
```

- /active:yes/no: Activa o desactiva la cuenta.
- /passwordchg:yes/no: Permite o impide que el usuario cambie la contraseña.
- /expires:fecha: Establece la fecha de expiración de la cuenta.

Asignar usuario a grupo

```
net localgroup Administradores nombre_usuario /add
```

```
net localgroup Usuarios nombre_usuario /delete
```

- Añade o elimina usuarios de grupos locales para gestionar permisos.

Ejemplo completo en PowerShell

```
New-LocalUser -Name "usuario1" -Password (ConvertTo-SecureString  
"ContraseñaSegura123!" -AsPlainText -Force) -FullName "Usuario de Prueba" -  
Description "Cuenta para pruebas" -PasswordNeverExpires $true
```

```
Add-LocalGroupMember -Group "Administradores" -Member "usuario1"
```

Políticas de contraseñas en sistemas operativos

Las políticas de contraseñas son reglas que definen los requisitos mínimos para la creación y mantenimiento de contraseñas seguras. Estas políticas se configuran para evitar contraseñas débiles y proteger las cuentas de accesos no autorizados.

Políticas en Windows

En Windows, las políticas de contraseñas se gestionan principalmente a través de las Directivas de Seguridad Local o mediante directivas de grupo (GPO) en entornos de Active Directory.

Las configuraciones más comunes incluyen:

- **Longitud mínima de contraseña:** Por ejemplo, 12 caracteres.
- **Complejidad:** Requiere combinación de mayúsculas, minúsculas, números y símbolos.
- **Historial de contraseñas:** Impide reutilizar las últimas N contraseñas.
- **Duración máxima:** Tiempo máximo que una contraseña puede ser usada antes de requerir cambio.
- **Duración mínima:** Tiempo mínimo antes de poder cambiar la contraseña.
- **Bloqueo de cuenta:** Número de intentos fallidos antes de bloquear la cuenta temporalmente.

Para configurar estas políticas desde línea de comandos se puede usar net accounts:

```
net accounts /minpwlen:12 /maxpwage:30 /minpwage:1 /uniquepw:5  
/lockoutthreshold:5 /lockoutduration:30
```

- /minpwlen:12 establece longitud mínima de 12 caracteres.
- /maxpwage:30 la contraseña expira a los 30 días.
- /minpwage:1 no se puede cambiar la contraseña antes de 1 día.
- /uniquepw:5 no se pueden reutilizar las últimas 5 contraseñas.
- /lockoutthreshold:5 bloquea la cuenta tras 5 intentos fallidos.
- /lockoutduration:30 duración del bloqueo en minutos.

Políticas en Linux

En Linux, las políticas de contraseñas se configuran mediante el archivo /etc/login.defs y herramientas como pam_pwquality o pam_tally2.

Ejemplo de configuración en /etc/login.defs:

```
PASS_MAX_DAYS 90
```


PASS_MIN_DAYS 7

PASS_MIN_LEN 12

PASS_WARN_AGE 14

Para configurar la complejidad y bloqueo con PAM:

- Editar /etc/pam.d/common-password para incluir:

```
password requisite pam_pwquality.so retry=3 minlen=12 dcredit=-1 ucredit=-1  
ocredit=-1 lcredit=-1
```

- Para bloqueo tras intentos fallidos, usar /etc/pam.d/common-auth:

```
auth required pam_tally2.so deny=5 unlock_time=900
```

Esto bloquea la cuenta tras 5 intentos fallidos durante 15 minutos.

Comandos para auditar usuarios y contraseñas

Auditar las cuentas y contraseñas es fundamental para detectar vulnerabilidades y mantener la seguridad.

En Windows

- Listar usuarios locales:

```
net user
```

- Ver detalles de un usuario:

```
net user nombre_usuario
```

- Comprobar políticas de contraseñas:

```
net accounts
```

- Revisar eventos de inicio de sesión fallidos (requiere PowerShell avanzado o visor de eventos).

En Linux

- Listar usuarios:

```
cut -d: -f1 /etc/passwd
```

- Ver grupos de un usuario:

```
groups nombre_usuario
```

- Comprobar fecha de expiración de contraseña:

```
chage -l nombre_usuario
```

- Revisar intentos fallidos de login:

```
sudo pam_tally2 --user nombre_usuario
```

- Auditar contraseñas débiles (requiere herramientas externas como john o hashcat).

Comandos adicionales para gestión y auditoría en Linux

Forzar cambio de contraseña en el próximo inicio de sesión

Para obligar a un usuario a cambiar su contraseña en el próximo inicio:

```
sudo chage -d 0 nombre_usuario
```

Esto establece la fecha del último cambio de contraseña a "0", lo que fuerza el cambio.

Bloquear y desbloquear cuentas de usuario

- Bloquear cuenta (impide inicio de sesión):

```
sudo usermod -L nombre_usuario
```

- Desbloquear cuenta:

```
sudo usermod -U nombre_usuario
```

- Alternativamente, se puede bloquear la contraseña:

```
sudo passwd -l nombre_usuario
```

- Y desbloquearla:

```
sudo passwd -u nombre_usuario
```

Ver usuarios con contraseñas expiradas

Para listar usuarios cuya contraseña ha expirado:

```
sudo chage -l nombre_usuario | grep 'Password expired'
```

Para automatizar y listar todos los usuarios con contraseñas expiradas:

```
for user in $(cut -f1 -d: /etc/passwd); do  
  
    if sudo chage -l $user | grep -q 'Password expired'; then  
  
        echo "Usuario con contraseña expirada: $user"  
  
    fi  
  
done
```

Script básico para auditar usuarios con contraseñas próximas a expirar

```
#!/bin/bash  
  
# Script para listar usuarios con contraseña a punto de expirar en menos de 7 días  
  
echo "Usuarios con contraseña a punto de expirar en menos de 7 días:"  
  
today=$(date +%s)  
  
  
for user in $(cut -f1 -d: /etc/passwd); do  
  
    expire_date=$(sudo chage -l $user | grep 'Password expires' | cut -d: -f2 | xargs)  
  
    if [[ "$expire_date" != "never" ]]; then  
  
        expire_sec=$(date -d "$expire_date" +%s)  
  
        diff_days=$(( (expire_sec - today) / 86400 ))  
  
        if (( diff_days >= 0 && diff_days <= 7 )); then  
  
            echo "$user - Expira en $diff_days días"  
  
        fi  
  
    fi  
  
done
```

done

Este script recorre todos los usuarios y muestra aquellos cuya contraseña expirará en menos de 7 días.

Ejemplo práctico: Configuración de bloqueo tras intentos fallidos con pam_tally2

Para proteger el sistema contra ataques de fuerza bruta, se puede configurar PAM para bloquear cuentas tras varios intentos fallidos.

1. Editar /etc/pam.d/common-auth y añadir:

```
auth required pam_tally2.so deny=5 unlock_time=900 onerr=fail audit
```

- deny=5: Bloquea tras 5 intentos fallidos.
- unlock_time=900: Desbloquea automáticamente tras 900 segundos (15 minutos).
- onerr=fail: En caso de error, falla la autenticación.
- audit: Registra eventos en logs.

2. Para ver el estado de bloqueo:

```
sudo pam_tally2 --user nombre_usuario
```

3. Para resetear el contador de intentos fallidos:

```
sudo pam_tally2 --user nombre_usuario --reset
```

Gestión avanzada de contraseñas y usuarios en Windows con PowerShell

PowerShell permite automatizar y gestionar usuarios y contraseñas con comandos y scripts avanzados, facilitando tareas complejas y masivas.

Crear usuario con contraseña segura y opciones avanzadas

```
$Password = ConvertTo-SecureString "ContraseñaSegura!2025" -AsPlainText -Force
```

New-LocalUser -Name "usuario1" -Password \$Password -FullName "Usuario de Prueba" -Description "Cuenta para pruebas" -PasswordNeverExpires \$false -UserMayNotChangePassword \$false

- -PasswordNeverExpires \$false: La contraseña expirará según la política.
- -UserMayNotChangePassword \$false: El usuario puede cambiar la contraseña.

Forzar cambio de contraseña en el próximo inicio

Set-LocalUser -Name "usuario1" -PasswordExpired \$true

Esto obliga al usuario a cambiar la contraseña en el siguiente inicio de sesión.

Modificar usuario: agregar a grupo local

Add-LocalGroupMember -Group "Administradores" -Member "usuario1"

Bloquear y desbloquear cuenta de usuario

- Bloquear cuenta:

Disable-LocalUser -Name "usuario1"

- Desbloquear cuenta:

Enable-LocalUser -Name "usuario1"

Cambiar contraseña de usuario

Set-LocalUser -Name "usuario1" -Password (ConvertTo-SecureString "NuevaContraseña!2025" -AsPlainText -Force)

Auditoría de usuarios y contraseñas con PowerShell

Listar todos los usuarios locales

Get-LocalUser

Ver detalles de un usuario específico

Get-LocalUser -Name "usuario1" | Format-List *

Comprobar si la contraseña está expirada

```
(Get-LocalUser -Name "usuario1").PasswordExpired
```

Listar usuarios con contraseña expirada (requiere módulo ActiveDirectory en dominio)

```
Search-ADAccount -PasswordExpired
```

Script para forzar cambio de contraseña a todos los usuarios que no la hayan cambiado en 90 días

```
Import-Module ActiveDirectory
```

```
$users = Get-ADUser -Filter * -Properties PasswordLastSet
```

```
foreach ($user in $users) {
```

```
    $pwdLastSet = $user.PasswordLastSet
```

```
    if ($pwdLastSet -lt (Get-Date).AddDays(-90)) {
```

```
        Set-ADUser -Identity $user.SamAccountName -ChangePasswordAtLogon $true
```

```
        Write-Output "Contraseña expirada forzada para $($user.SamAccountName)"
```

```
    }
```

```
}
```

6.2. Permisos de acceso y directivas locales

Introducción

El control de acceso es un componente esencial en la seguridad informática, ya que determina qué usuarios o procesos pueden acceder a recursos específicos dentro de un sistema o red. Los permisos de acceso regulan la capacidad de leer, modificar, ejecutar o administrar archivos, carpetas, dispositivos y servicios, garantizando que solo los usuarios autorizados puedan realizar acciones determinadas.

Las directivas locales son configuraciones que aplican reglas de seguridad y políticas en un equipo o servidor, permitiendo definir comportamientos específicos para usuarios y grupos. Estas directivas pueden incluir restricciones de acceso, políticas de auditoría, configuraciones de contraseñas, y otras medidas que fortalecen la seguridad del sistema.

En este apartado se profundizará en los diferentes tipos de permisos y modelos de control de acceso, cómo se gestionan en sistemas Windows y Linux, y cómo configurar y administrar directivas locales para mantener un entorno seguro y controlado. Se presentarán comandos y herramientas para la gestión avanzada, así como ejemplos prácticos y tablas comparativas para facilitar la comprensión.

Modelos y tipos de permisos de acceso

Los modelos de control de acceso más comunes son:

1. **Control de acceso discrecional (DAC)**
El propietario del recurso define quién puede acceder y qué tipo de acceso se permite. Es flexible pero puede ser menos seguro si no se administra correctamente.
2. **Control de acceso obligatorio (MAC)**
El sistema impone políticas estrictas basadas en niveles de seguridad y etiquetas, sin que el propietario pueda modificarlas. Se usa en entornos altamente seguros.
3. **Control de acceso basado en roles (RBAC)**
Los permisos se asignan a roles específicos, y los usuarios adquieren permisos al ser asignados a esos roles. Facilita la gestión en entornos grandes.

Permisos en sistemas Windows

En Windows, los permisos se gestionan principalmente a través del sistema NTFS y las directivas locales o de grupo.

Tipos de permisos NTFS básicos

- **Control total:** Permite todas las acciones, incluyendo cambiar permisos y tomar posesión.
- **Modificar:** Permite leer, escribir, ejecutar y eliminar archivos.
- **Lectura y ejecución:** Permite ver y ejecutar archivos.
- **Lectura:** Permite ver el contenido.
- **Escritura:** Permite modificar el contenido.

Permisos avanzados

Windows permite configurar permisos avanzados como:

- Tomar posesión
- Cambiar permisos
- Listar carpetas
- Eliminar subcarpetas y archivos

Herramientas para gestionar permisos

- **Explorador de archivos:** Interfaz gráfica para asignar permisos.
- **icacls:** Comando para ver y modificar permisos desde línea de comandos.
- **secedit:** Para aplicar plantillas de seguridad.
- **gpedit.msc:** Editor de directivas locales.

Ejemplo de uso de icacls

icacls "C:\\Carpeta" /grant Usuario:(OI)(CI)F /T

- (OI): Heredar permisos a archivos.
- (CI): Heredar permisos a carpetas.
- F: Control total.
- /T: Aplicar recursivamente.

Permisos en sistemas Linux

Linux utiliza un modelo de permisos basado en propietario, grupo y otros, con tres tipos básicos de permisos:

- **Lectura (r):** Permite leer el contenido.
- **Escritura (w):** Permite modificar el contenido.
- **Ejecución (x):** Permite ejecutar archivos o acceder a directorios.

Representación de permisos

Los permisos se representan con una cadena de 10 caracteres, por ejemplo:

- El primer carácter indica tipo de archivo.
- Los siguientes tres son permisos del propietario.
- Los siguientes tres son permisos del grupo.
- Los últimos tres son permisos para otros usuarios.

Comandos para gestionar permisos

- `chmod`: Cambiar permisos.
- `chown`: Cambiar propietario.
- `chgrp`: Cambiar grupo.
- `getfacl` y `setfacl`: Gestionar listas de control de acceso (ACLs) para permisos más granulares.

Ejemplo de uso de `chmod`

`chmod 750 archivo.sh`

- 7 (rwx) para propietario.
- 5 (r-x) para grupo.
- 0 (---) para otros.

Uso de ACLs

Para añadir un permiso específico a un usuario:

`setfacl -m u:usuario:rwx archivo.sh`

Para ver permisos ACL:

`getfacl archivo.sh`

Tabla comparativa de gestión de permisos Windows vs Linux

Aspecto	Windows (NTFS)	Linux (Permisos tradicionales y ACL)

Modelo de permisos	Basado en ACL con permisos detallados	Propietario, grupo, otros + ACLs opcionales
Permisos básicos	Control total, modificar, lectura, escritura	Lectura, escritura, ejecución
Herramientas gestión	icacls, secedit, gpedit.msc	chmod, chown, chgrp, getfacl, setfacl
Herencia de permisos	Sí, configurable con flags (OI, CI)	Sí, con ACLs y permisos por defecto
Control granular	Muy granular, con permisos avanzados	ACLs permiten granularidad adicional
Gestión gráfica	Explorador de archivos, directivas locales	Gestores de archivos con GUI (Nautilus, Dolphin)

Ejemplos prácticos y comandos avanzados para permisos y directivas locales

Windows: Gestión avanzada de permisos con icacls

El comando icacls permite ver, modificar y respaldar permisos NTFS desde la línea de comandos.

- **Ver permisos de un archivo o carpeta:**

```
icacls "C:\\Ruta\\Archivo_o_Carpeta"
```

- **Conceder control total a un usuario, aplicando herencia a archivos y carpetas:**

```
icacls "C:\\Ruta\\Carpeta" /grant Usuario:(OI)(CI)F /T
```

- **Quitar todos los permisos a un usuario:**

```
icacls "C:\\Ruta\\Carpeta" /remove Usuario /T
```

- **Respalda permisos de una carpeta a un archivo:**

```
icacls "C:\\Ruta\\Carpeta" /save permisos_backup.txt /T
```

- **Restaurar permisos desde un archivo de respaldo:**

```
icacls "C:\\Ruta\\Carpeta" /restore permisos_backup.txt
```

Windows: Configuración de directivas locales con secedit y gpedit.msc

- **Exportar configuración de seguridad local a un archivo INF:**

```
secedit /export /cfg C:\\backup\\secpol.inf
```

- **Importar y aplicar configuración desde un archivo INF:**

```
secedit /configure /db secedit.sdb /cfg C:\\backup\\secpol.inf /overwrite
```

- **Abrir editor de directivas locales (GUI):**

```
gpedit.msc
```

Desde aquí se pueden configurar políticas como:

- Restricciones de acceso a carpetas.
- Políticas de auditoría.
- Configuración de contraseñas y bloqueo de cuentas.
- Configuración de scripts de inicio y cierre de sesión.

Linux: Gestión avanzada de permisos con chmod, chown y ACLs

- **Cambiar permisos recursivamente:**

```
sudo chmod -R 750 /ruta/carpeta
```

- **Cambiar propietario y grupo recursivamente:**

```
sudo chown -R usuario:grupo /ruta/carpeta
```

- **Añadir permisos ACL para un usuario específico:**

```
sudo setfacl -m u:usuario:rwx /ruta/archivo_o_carpeta
```

- **Eliminar permisos ACL para un usuario:**

```
sudo setfacl -x u:usuario /ruta/archivo_o_carpeta
```

- **Ver permisos ACL:**

```
getfacl /ruta/archivo_o_carpeta
```

Linux: Configuración de directivas locales y seguridad con PAM y SELinux/AppArmor

- **Configurar restricciones de acceso mediante PAM**

Ejemplo para limitar acceso SSH solo a usuarios de un grupo:

Editar `/etc/security/access.conf` y añadir:

```
 -:ALL EXCEPT grupo_ssh:ALL
```

Luego en `/etc/pam.d/sshd` añadir:

```
account required pam_access.so
```

- **Uso de SELinux para control de acceso obligatorio (MAC)**

SELinux aplica políticas estrictas que controlan qué procesos pueden acceder a qué recursos.

- Comandos básicos:

```
sestatus      # Ver estado de SELinux
```

```
getenforce    # Ver modo actual (Enforcing, Permissive, Disabled)
```

```
setenforce 1   # Activar modo enforcing
```

```
setenforce 0   # Desactivar modo enforcing temporalmente
```

- **AppArmor** es otra solución MAC usada en distribuciones como Ubuntu.

Tabla resumen de comandos clave para permisos y directivas

Sistema	Comando / Herramienta	Función
Windows	icacls	Gestionar permisos NTFS
Windows	secedit	Exportar/importar configuración de seguridad
Windows	gpedit.msc	Editor de directivas locales (GUI)
Linux	chmod, chown, chgrp	Cambiar permisos, propietario y grupo
Linux	setfacl, getfacl	Gestionar y consultar ACLs
Linux	pam_access.so	Control de acceso mediante PAM
Linux	sestatus, getenforce	Gestión de SELinux

6.3 Control de procesos y servicios del sistema

El control de procesos y servicios es una función crítica en la administración de sistemas operativos, ya que permite gestionar la ejecución de programas y servicios que mantienen el sistema operativo y las aplicaciones funcionando correctamente. Un proceso es una instancia de un programa en ejecución, que consume recursos como CPU, memoria y acceso a dispositivos. Los servicios, por otro lado, son procesos que se ejecutan en segundo plano para proporcionar funcionalidades específicas, como servidores web, bases de datos, o servicios de red.

Desde el punto de vista teórico, un sistema operativo debe gestionar múltiples procesos simultáneamente, garantizando la asignación eficiente de recursos y la protección entre procesos. Esto se logra mediante la planificación de procesos (scheduling), gestión de memoria, y mecanismos de comunicación entre procesos (IPC). Los servicios, como procesos de larga duración, deben ser gestionados para asegurar su disponibilidad y estabilidad, lo que implica mecanismos de inicio, parada, supervisión y recuperación ante fallos.

El modelo de procesos incluye estados (nuevo, listo, ejecutando, bloqueado, terminado) y transiciones que el sistema operativo controla para optimizar el rendimiento y la respuesta. La gestión de servicios implica además políticas de inicio (automático, manual, deshabilitado) y dependencias entre servicios.

Este apartado profundiza en los conceptos fundamentales de procesos y servicios, las herramientas y comandos para su gestión en Windows y Linux, técnicas para monitorizar y controlar su comportamiento, y buenas prácticas para mantener un sistema seguro y eficiente.

6.3.1 Conceptos fundamentales de procesos y servicios

6.3.1.1 Procesos

Un proceso es la unidad básica de ejecución en un sistema operativo. Teóricamente, un proceso consta de:

- **Código ejecutable:** El programa en sí.
- **Contexto de proceso:** Incluye registros de CPU, contador de programa, pila y memoria asignada.
- **Estado:** Indica la fase en la que se encuentra el proceso (ejecución, espera, suspendido, etc.).
- **Identificador único (PID):** Para distinguirlo de otros procesos.

Los procesos pueden ser **independientes** o **cooperativos**, y pueden comunicarse mediante mecanismos IPC (pipes, colas, memoria compartida). La planificación de procesos es un problema clásico en sistemas operativos, con algoritmos como Round Robin, Prioridad, FIFO, entre otros.

Además, la gestión de procesos debe considerar la sincronización y comunicación entre procesos para evitar condiciones de carrera, deadlocks y garantizar la integridad de los datos.

6.3.1.2 Hilos (Threads)

Los hilos son subunidades de un proceso que permiten la ejecución concurrente dentro del mismo espacio de memoria. La teoría de hilos se basa en la concurrencia y paralelismo, mejorando la eficiencia y capacidad de respuesta de las aplicaciones. La sincronización entre hilos es crítica para evitar condiciones de carrera y garantizar la integridad de datos.

Los sistemas operativos modernos soportan multihilo a nivel de usuario y kernel, permitiendo que múltiples hilos se ejecuten en paralelo en procesadores multinúcleo.

6.3.1.3 Servicios y demonios

Los servicios (Windows) y demonios (Linux) son procesos diseñados para ejecutarse en segundo plano, proporcionando funcionalidades continuas sin interacción directa con el usuario. Teóricamente, estos procesos deben ser gestionados para garantizar su disponibilidad, seguridad y rendimiento, mediante políticas de inicio, supervisión y recuperación.

Los servicios suelen ejecutarse con privilegios limitados para minimizar riesgos de seguridad y deben ser configurados para reiniciarse automáticamente en caso de fallo, asegurando la resiliencia del sistema.

6.3.2 Gestión de procesos en Windows

6.3.2.1 Introducción

La gestión de procesos en Windows es una tarea fundamental para el mantenimiento y la seguridad del sistema operativo. Los procesos representan programas en ejecución que consumen recursos del sistema, y su control adecuado permite optimizar el rendimiento, detectar comportamientos anómalos y garantizar la estabilidad del entorno. Windows ofrece diversas herramientas gráficas y de línea de comandos para visualizar, controlar y administrar estos procesos, facilitando tanto la intervención manual como la automatización mediante scripts.

6.3.2.2 Herramientas gráficas

Windows incluye herramientas visuales que permiten a los usuarios y administradores monitorizar y gestionar procesos de forma intuitiva:

- **Administrador de tareas (Task Manager):** Proporciona una vista en tiempo real de los procesos activos, uso de CPU, memoria, disco y red. Permite finalizar procesos, cambiar su prioridad y analizar el rendimiento general del sistema.
- **Monitor de recursos (Resource Monitor):** Ofrece información detallada sobre el uso de recursos por parte de procesos específicos, incluyendo CPU, disco, red y memoria, facilitando la identificación de cuellos de botella o procesos problemáticos.

Estas herramientas son esenciales para la supervisión diaria y permiten una rápida intervención ante procesos que consumen recursos excesivos o que se comportan de forma anómala.

6.3.2.3 Comandos y PowerShell

PowerShell es un shell orientado a objetos que permite la automatización y gestión avanzada de procesos. Los cmdlets como Get-Process, Stop-Process y Start-Process permiten manipular procesos programáticamente, facilitando tareas de administración y scripting.

- Listar procesos:

Get-Process

- Filtrar procesos por nombre:

Get-Process -Name chrome

- Obtener información detallada de un proceso:

Get-Process -Id 1234 | Format-List *

- Detener proceso por nombre o PID:

Stop-Process -Name notepad

Stop-Process -Id 1234

- Iniciar proceso:

Start-Process notepad.exe

Ventajas:

El uso de PowerShell para gestionar procesos permite automatizar tareas repetitivas, integrar controles en scripts de mantenimiento y reaccionar rápidamente a condiciones anómalas sin necesidad de intervención manual. Además, la capacidad de filtrar y ordenar procesos facilita la identificación de cuellos de botella y procesos maliciosos.

6.3.2.4 Monitorización avanzada

- Uso de Get-Process con filtros para procesos que consumen mucha CPU:

Get-Process | Where-Object {\$_.CPU -gt 100}

- Obtener procesos ordenados por uso de memoria:

Get-Process | Sort-Object -Property WS -Descending

Ventajas:

Estas técnicas permiten focalizar la atención en procesos que impactan significativamente en el rendimiento, facilitando la toma de decisiones para optimizar recursos o investigar posibles problemas.

6.3.3 Gestión de servicios en Windows

6.3.3.1 Introducción

Los servicios en Windows son procesos gestionados por el Service Control Manager (SCM), que controla su ciclo de vida y estado. La teoría detrás de los servicios incluye la necesidad de procesos persistentes que proporcionen funcionalidades esenciales, con mecanismos para iniciar, detener, pausar y reiniciar servicios según sea necesario.

La correcta gestión de servicios es vital para la seguridad y estabilidad del sistema, ya que servicios innecesarios o mal configurados pueden ser vectores de ataque o consumir recursos innecesariamente.

6.3.3.2 Comandos PowerShell para servicios

Los cmdlets de PowerShell para servicios permiten gestionar el estado y configuración de los mismos, integrándose con el modelo de administración basado en objetos de PowerShell.

- Listar todos los servicios:

`Get-Service`

- Filtrar servicios en ejecución:

`Get-Service | Where-Object {$_.Status -eq 'Running'}`

- Iniciar, detener o reiniciar un servicio:

`Start-Service -Name wuauserv`

`Stop-Service -Name wuauserv`

`Restart-Service -Name wuauserv`

- Cambiar tipo de inicio:

`Set-Service -Name wuauserv -StartupType Automatic`

```
Set-Service -Name wuauserv -StartupType Manual
```

```
Set-Service -Name wuauserv -StartupType Disabled
```

6.3.3.3 Ejemplo: Script para reiniciar un servicio si está detenido

```
$service = Get-Service -Name wuauserv
```

```
if ($service.Status -ne 'Running') {
```

```
    Start-Service -Name wuauserv
```

```
    Write-Output "Servicio wuauserv iniciado."
```

```
} else {
```

```
    Write-Output "Servicio wuauserv ya está en ejecución."
```

```
}
```

Ventajas:

Automatizar la supervisión y reinicio de servicios críticos garantiza la alta disponibilidad y reduce el tiempo de inactividad, evitando la necesidad de intervención manual constante y mejorando la resiliencia del sistema.

6.3.4 Gestión de procesos en Linux

6.3.4.1 Introducción

En Linux, la gestión de procesos es una tarea esencial para la administración del sistema. Los procesos pueden ser monitorizados y controlados mediante diversas herramientas de línea de comandos que permiten obtener información detallada, modificar prioridades y terminar procesos. La flexibilidad y potencia de estas herramientas facilitan la administración eficiente y la resolución de problemas.

6.3.4.2 Herramientas de monitorización

- **top:** Monitor en tiempo real de procesos y recursos.
- **htop:** Versión mejorada de top con interfaz interactiva y colores.
- **ps:** Listar procesos con diferentes opciones.

6.3.4.3 Comandos básicos

- Listar todos los procesos:

```
ps aux
```

- Buscar procesos por nombre:

```
ps aux | grep apache2
```

- Terminar proceso por PID:

```
kill 1234
```

- Forzar terminación:

```
kill -9 1234
```

- Terminar proceso por nombre:

```
killall apache2
```

Ventajas:

El uso de estas herramientas permite a los administradores identificar procesos problemáticos, liberar recursos y mantener la estabilidad del sistema. La capacidad de enviar señales específicas a procesos ofrece un control granular sobre su comportamiento.

6.3.4.4 Monitorización avanzada

- Ver procesos ordenados por uso de CPU:

```
ps aux --sort=-%cpu | head -n 10
```

- Ver procesos ordenados por uso de memoria:

```
ps aux --sort=-%mem | head -n 10
```

Ventajas:

Ordenar procesos por consumo de recursos facilita la detección de cuellos de botella y procesos que podrían estar afectando el rendimiento general, permitiendo una intervención rápida y eficaz.

6.3.5 Gestión de servicios en Linux (systemd)

6.3.5.1 Introducción

systemd es el sistema de inicio y gestor de servicios más común en distribuciones modernas de Linux. Permite una gestión eficiente y paralela de servicios, mejorando la velocidad de arranque y la administración de dependencias entre servicios. La gestión de servicios es fundamental para mantener la estabilidad y seguridad del sistema.

6.3.5.2 Comandos básicos

- Listar servicios activos:

```
systemctl list-units --type=service --state=running
```

```
systemctl status sshd
```

- Iniciar, detener, reiniciar servicio:

```
sudo systemctl start sshd
```

```
sudo systemctl stop sshd
```

```
sudo systemctl restart sshd
```

- Habilitar o deshabilitar servicio para inicio automático:

```
sudo systemctl enable sshd
```

```
sudo systemctl disable sshd
```

6.3.5.3 Ejemplo: Crear un servicio personalizado

Archivo `/etc/systemd/system/mi_servicio.service`:

```
[Unit]
```

```
Description=Mí Servicio Personalizado
```

```
[Service]
```

```
ExecStart=/usr/bin/python3 /home/usuario/mi_script.py
```

```
Restart=always
```

```
[Install]
```

WantedBy=multi-user.target

Comandos para habilitar y arrancar:

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable mi_servicio
```

```
sudo systemctl start mi_servicio
```

Ventajas:

Crear servicios personalizados con systemd permite integrar aplicaciones propias en el sistema de gestión de servicios, asegurando su arranque automático, supervisión y reinicio en caso de fallo, lo que mejora la robustez y disponibilidad.

6.3.6 Monitorización y control avanzado

6.3.6.1 Renice y prioridad de procesos

La prioridad de procesos determina el orden y cantidad de tiempo CPU asignado. El sistema operativo usa esta información para planificar la ejecución, optimizando el rendimiento y la respuesta.

Cambiar la prioridad de un proceso en Linux:

```
sudo renice -n 10 -p 1234
```

Valores de prioridad van de -20 (más prioridad) a 19 (menos prioridad).

Ventajas:

Ajustar la prioridad permite optimizar el uso de CPU, dando preferencia a procesos críticos y relegando procesos menos importantes, mejorando la eficiencia general del sistema.

6.3.6.2 Scripts para monitorizar procesos y reiniciar servicios

La automatización mediante scripts permite mantener la disponibilidad y estabilidad del sistema, reaccionando ante fallos o condiciones anómalas.

Ejemplo de script para reiniciar un servicio si no está activo:

```
#!/bin/bash
```

```
SERVICE="apache2"

if ! systemctl is-active --quiet $SERVICE; then

    systemctl start $SERVICE

    echo "Servicio $SERVICE iniciado."

else

    echo "Servicio $SERVICE ya está activo."

fi
```

Ventajas:

Este enfoque automatizado reduce la necesidad de supervisión manual constante, minimiza tiempos de inactividad y asegura que servicios críticos estén siempre disponibles, mejorando la resiliencia del sistema.

6.3.7 Buenas prácticas para control de procesos y servicios

- Mantener solo los servicios necesarios activos para reducir la superficie de ataque.
- Monitorizar el uso de recursos para detectar procesos anómalos.
- Configurar reinicio automático de servicios críticos en caso de fallo.
- Usar cuentas con privilegios mínimos para ejecutar servicios.
- Auditar periódicamente los procesos y servicios en ejecución.
- Documentar configuraciones y cambios realizados.
- Implementar políticas de seguridad para limitar la ejecución de procesos no autorizados.
- Utilizar herramientas de monitorización centralizada para entornos empresariales.

6.4 Herramientas de optimización y diagnóstico

6.4 Introducción

La optimización y diagnóstico de sistemas informáticos son actividades esenciales para garantizar el rendimiento, estabilidad y seguridad de los entornos tecnológicos. En el contexto de la informática avanzada, disponer de herramientas adecuadas para monitorizar, analizar y mejorar el funcionamiento de hardware y software es fundamental para anticipar problemas, resolver incidencias y mantener la eficiencia operativa.

Las herramientas de optimización permiten identificar cuellos de botella, gestionar recursos de manera eficiente y mejorar la experiencia del usuario final. Por otro lado, las herramientas de diagnóstico facilitan la detección de fallos, análisis de logs, evaluación del estado de componentes y verificación de configuraciones, contribuyendo a una administración proactiva y reactiva del sistema.

Este apartado aborda las principales herramientas disponibles en sistemas Windows y Linux para optimizar y diagnosticar sistemas, incluyendo utilidades gráficas y de línea de comandos, técnicas avanzadas de análisis, ejemplos prácticos y buenas prácticas para su uso efectivo.

6.4.1 Herramientas de optimización en Windows

6.4.1.1 Administrador de tareas (Task Manager)

El Administrador de tareas es una herramienta gráfica fundamental para monitorizar el uso de CPU, memoria, disco, red y GPU en tiempo real. Permite identificar procesos que consumen recursos excesivos y finalizar tareas problemáticas.

- **Pestañas clave:**
 - *Procesos*: Lista de aplicaciones y procesos en ejecución con uso de recursos.
 - *Rendimiento*: Gráficos en tiempo real del uso de CPU, memoria, disco, red y GPU.
 - *Historial de aplicaciones*: Uso de recursos por aplicación a lo largo del tiempo.
 - *Inicio*: Gestión de programas que se ejecutan al iniciar Windows.
 - *Usuarios*: Recursos consumidos por cada usuario conectado.

Ventajas:

Permite una rápida identificación y resolución de problemas de rendimiento sin necesidad de herramientas externas.

6.4.1.2 Monitor de recursos (Resource Monitor)

El Monitor de recursos ofrece un análisis más detallado que el Administrador de tareas, mostrando información sobre CPU, disco, red y memoria con desglose por proceso y servicio.

- **Características destacadas:**

- Visualización de procesos que acceden a disco y archivos abiertos.
- Monitorización de conexiones de red activas y puertos utilizados.
- Análisis de uso de memoria física y virtual.

Ventajas:

Facilita la identificación precisa de procesos que consumen recursos y permite diagnosticar problemas complejos relacionados con I/O y red.

6.4.1.3 Liberador de espacio en disco (Disk Cleanup)

Esta utilidad elimina archivos temporales, cachés, archivos de sistema y otros elementos innecesarios para liberar espacio en disco y mejorar el rendimiento.

- **Opciones avanzadas:**

- Limpieza de archivos de actualización de Windows.
- Eliminación de puntos de restauración antiguos.
- Limpieza de archivos temporales de usuario.

Ventajas:

Mantener espacio libre en disco es crucial para el rendimiento y estabilidad del sistema, especialmente en discos SSD.

6.4.1.4 Desfragmentador de disco

Optimiza la disposición física de los archivos en discos HDD para mejorar los tiempos de acceso y lectura.

- **Nota:** No es necesario en discos SSD, donde la desfragmentación puede reducir la vida útil.

6.4.1.5 Comandos para optimización y diagnóstico

- **perfmon (Monitor de rendimiento):** Permite crear contadores personalizados para monitorizar métricas específicas del sistema.

perfmon

- **sfc /scannow:** Escanea y repara archivos de sistema corruptos, manteniendo la integridad del sistema operativo.

sfc /scannow

- **chkdsk:** Verifica y repara errores en discos, mejorando la fiabilidad del almacenamiento.

chkdsk C: /f /r

6.4.1.6 Ejemplo: Uso de PowerShell para monitorizar procesos y recursos

Get-Process | Sort-Object CPU -Descending | Select-Object -First 10

Ventajas:

Permite identificar rápidamente los procesos que consumen más CPU, facilitando la toma de decisiones para optimizar el rendimiento.

6.4.2 Herramientas de optimización en Linux

6.4.2.1 Monitorización de recursos

- **top:** Muestra en tiempo real los procesos activos y el uso de CPU, memoria y swap. Permite ordenar por diferentes columnas y enviar señales a procesos.

top

- **htop:** Versión mejorada de top con interfaz interactiva, colores y navegación con teclado.

htop

- **vmstat:** Proporciona estadísticas sobre memoria, procesos, CPU y sistema de entrada/salida, útil para análisis de rendimiento a largo plazo.

vmstat 5

6.4.2.2 Análisis de disco y sistema de archivos

- **df:** Muestra el uso del espacio en disco de forma legible.

df -h

- **du:** Calcula el tamaño de directorios y archivos, útil para identificar directorios que consumen mucho espacio.

du -sh /ruta/directorio

- **iostat:** Proporciona estadísticas detalladas de entrada/salida de dispositivos, ayudando a detectar cuellos de botella en discos.

iostat -x 5

6.4.2.3 Diagnóstico de red

- **netstat:** Muestra conexiones de red, tablas de enrutamiento y estadísticas.

netstat -tulnp

- **ss:** Herramienta moderna para mostrar sockets y conexiones de red.

ss -tulw

- **ping y traceroute:** Diagnóstico básico de conectividad y rutas de red.

ping google.com

traceroute google.com

6.4.2.4 Análisis de logs

- **journalctl:** Visualiza logs del sistema gestionados por systemd, con opciones para filtrar por servicio, tiempo o nivel de severidad.

journalctl -xe

- **tail y grep:** Para analizar archivos de log tradicionales, permitiendo seguimiento en tiempo real y búsqueda de patrones.

```
tail -f /var/log/syslog
```

```
grep error /var/log/syslog
```

6.4.2.5 Ejemplo: Script para monitorizar uso de CPU y enviar alerta

```
#!/bin/bash
```

```
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 + $4}')
```

```
THRESHOLD=80
```

```
if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then
```

```
    echo "Alerta: Uso de CPU alto - $CPU_USAGE%" | mail -s "Alerta CPU"  
    admin@dominio.com
```

```
fi
```

Ventajas:

Automatiza la supervisión del sistema y notifica al administrador ante condiciones críticas, permitiendo una respuesta rápida y proactiva que puede evitar caídas o degradación del servicio.

6.4.3 Herramientas de diagnóstico avanzadas

6.4.3.1 Windows Performance Toolkit (WPT)

Conjunto de herramientas para análisis detallado de rendimiento, incluyendo:

- **Windows Performance Recorder (WPR):** Captura trazas de eventos del sistema y aplicaciones.
- **Windows Performance Analyzer (WPA):** Analiza y visualiza datos de rendimiento para identificar cuellos de botella y problemas complejos.

Ventajas:

Permite un análisis profundo y detallado del comportamiento del sistema, ideal para resolver problemas difíciles de diagnosticar con herramientas básicas.

6.4.3.2 Sysinternals Suite

Colección de utilidades avanzadas para diagnóstico y optimización, destacando:

- **Process Explorer:** Monitor avanzado de procesos con información detallada de handles y DLLs.
- **Autoruns:** Gestión exhaustiva de programas que se inician automáticamente.
- **TCPView:** Visualización en tiempo real de conexiones de red y puertos abiertos.

Ventajas:

Ofrece herramientas especializadas para análisis forense, detección de malware y optimización avanzada.

6.4.3.3 Herramientas de diagnóstico en Linux

- **strace:** Rastrea llamadas al sistema realizadas por un proceso, útil para depurar y entender comportamiento.

strace -p PID

- **lsof:** Lista archivos abiertos por procesos, ayudando a identificar recursos en uso.

lsof -p PID

- **perf:** Herramienta para análisis de rendimiento a nivel de CPU, incluyendo perfiles de uso y cuellos de botella.

perf top

6.4.4 Buenas prácticas para optimización y diagnóstico

- Realizar monitorización continua para detectar problemas antes de que afecten al usuario.
- Automatizar alertas y respuestas ante condiciones críticas.
- Mantener actualizado el software y controladores para evitar problemas de rendimiento.
- Documentar configuraciones y cambios realizados.
- Utilizar herramientas adecuadas para cada tipo de diagnóstico.
- Realizar análisis periódicos de logs y rendimiento.

- Priorizar la seguridad en la configuración para evitar que optimizaciones comprometan la integridad.
- Capacitar al personal en el uso de herramientas y análisis de datos para una administración efectiva.

6.5 Comunicaciones seguras: VPN, HTTPS, SSH

6.5 Introducción

En la era digital actual, la seguridad en las comunicaciones es fundamental para proteger la confidencialidad, integridad y autenticidad de la información transmitida a través de redes públicas y privadas. Las comunicaciones inseguras pueden ser interceptadas, modificadas o suplantadas, lo que pone en riesgo datos sensibles, credenciales y la privacidad de los usuarios.

Para mitigar estos riesgos, se utilizan tecnologías y protocolos que cifran la información y autentican a las partes involucradas, garantizando que solo los destinatarios legítimos puedan acceder a los datos y que estos no hayan sido alterados durante la transmisión. Entre las tecnologías más utilizadas para comunicaciones seguras destacan las VPN (Redes Privadas Virtuales), HTTPS (HTTP seguro) y SSH (Secure Shell).

Este apartado profundiza en el funcionamiento, configuración y ventajas de estas tecnologías, proporcionando ejemplos prácticos y comandos para su implementación en entornos Windows y Linux, así como buenas prácticas para mantener comunicaciones seguras.

6.5.1 VPN (Redes Privadas Virtuales)

6.5.1.1 Concepto y funcionamiento

Una VPN crea un túnel cifrado entre el dispositivo del usuario y una red privada, permitiendo que los datos viajen de forma segura a través de redes públicas como Internet. Esto protege la información contra espionaje y ataques de intermediarios (man-in-the-middle).

Las VPN pueden ser de varios tipos, incluyendo:

- **VPN de acceso remoto:** Permite a usuarios conectarse a una red privada desde ubicaciones remotas.
- **VPN de sitio a sitio:** Conecta dos redes privadas a través de Internet de forma segura.

6.5.1.2 Protocolos comunes

- **IPSec:** Protocolo estándar para cifrado y autenticación a nivel de red.
- **OpenVPN:** Solución flexible y segura basada en SSL/TLS.
- **WireGuard:** Protocolo moderno, eficiente y fácil de configurar.

6.5.1.3 Ejemplo de configuración básica OpenVPN en Linux

```
sudo apt-get install openvpn
```

```
sudo openvpn --config /ruta/al/archivo.ovpn
```

Opciones explicadas:

- `--config /ruta/al/archivo.ovpn`: Especifica el archivo de configuración que contiene los parámetros de conexión, certificados y claves.
- `--daemon`: Ejecuta OpenVPN en segundo plano.
- `--log /ruta/al/log.txt`: Guarda un registro detallado de la conexión para diagnóstico.
- `--auth-user-pass /ruta/credenciales.txt`: Permite usar un archivo con usuario y contraseña para autenticación.

Ventajas:

OpenVPN ofrece un alto nivel de seguridad, compatibilidad multiplataforma y flexibilidad para configuraciones avanzadas, siendo ampliamente utilizado en entornos corporativos y personales.

6.5.2 HTTPS (HTTP Secure)

6.5.2.1 Concepto y funcionamiento

HTTPS es la versión segura del protocolo HTTP, que utiliza TLS (Transport Layer Security) para cifrar la comunicación entre el navegador y el servidor web. Esto garantiza que los datos transmitidos, como credenciales y transacciones, estén protegidos contra interceptación y manipulación.

6.5.2.2 Certificados digitales

Para establecer una conexión HTTPS, el servidor debe presentar un certificado digital emitido por una autoridad certificadora (CA) confiable. Este certificado autentica la identidad del servidor y permite el intercambio seguro de claves.

6.5.2.3 Configuración básica en Apache (Linux)

```
sudo a2enmod ssl
```

```
sudo a2ensite default-ssl
```

```
sudo systemctl reload apache2
```

Opciones explicadas:

- `a2enmod ssl`: Activa el módulo SSL en Apache para habilitar soporte HTTPS.
- `a2ensite default-ssl`: Habilita el sitio de configuración SSL predeterminado.
- `systemctl reload apache2`: Recarga la configuración sin detener el servicio.

Para configurar certificados personalizados, se editan los archivos de configuración para apuntar a:

```
SSLCertificateFile /etc/ssl/certs/mi_certificado.crt
```

```
SSLCertificateKeyFile /etc/ssl/private/mi_clave.key
```

```
SSLCertificateChainFile /etc/ssl/certs/ca_bundle.crt
```

Ventajas:

HTTPS es esencial para proteger la privacidad y seguridad en la web, además de ser un factor clave para la confianza del usuario y el posicionamiento en buscadores.

6.5.3 SSH (Secure Shell)

6.5.3.1 Concepto y funcionamiento

SSH es un protocolo para acceder de forma segura a sistemas remotos mediante una conexión cifrada. Proporciona autenticación fuerte, integridad de datos y confidencialidad, siendo la herramienta estándar para administración remota en sistemas Unix/Linux.

6.5.3.2 Autenticación

SSH soporta autenticación mediante contraseña y, preferentemente, mediante claves públicas y privadas, que ofrecen mayor seguridad y facilitan la automatización.

6.5.3.3 Comandos básicos y opciones

- Conectar a un servidor remoto:

```
ssh usuario@servidor
```

Opciones comunes:

- -p puerto: Especifica un puerto distinto al 22 por defecto.

```
ssh -p 2222 usuario@servidor
```

- -i /ruta/clave_privada: Usa una clave privada específica para autenticación.

```
ssh -i ~/.ssh/id_rsa usuario@servidor
```

- -v, -vv, -vvv: Modo verbose para depuración.

```
ssh -vvv usuario@servidor
```

- Copiar archivos de forma segura:

```
scp archivo.txt usuario@servidor:/ruta/destino
```

Opciones comunes:

```
scp -r carpeta usuario@servidor:/ruta/destino
```

- -P puerto: Especificar puerto distinto para SCP.

```
scp -P 2222 archivo.txt usuario@servidor:/ruta/destino
```

- Generar par de claves SSH:

```
ssh-keygen -t rsa -b 4096
```

Opciones explicadas:

- -t rsa: Tipo de clave (RSA).

- -b 4096: Tamaño de la clave en bits (más bits, mayor seguridad).
- -C "comentario": Añade un comentario para identificar la clave.

Ventajas:

SSH permite una administración remota segura, evitando la exposición de credenciales y protegiendo la integridad de las sesiones. Su flexibilidad permite integrarlo en scripts y automatizaciones, facilitando la gestión de infraestructuras.

6.5.4 Buenas prácticas para comunicaciones seguras

- Utilizar siempre protocolos cifrados para transmisión de datos sensibles.
- Configurar autenticación fuerte, preferiblemente con claves públicas.
- Mantener actualizados los software y certificados para evitar vulnerabilidades.
- Limitar el acceso remoto mediante firewalls y listas blancas.
- Monitorizar y auditar conexiones para detectar accesos no autorizados.
- Educar a usuarios sobre riesgos y prácticas seguras.

6.6 Configuración de firewall (ufw, Windows Defender Firewall)

6.6 Introducción

Los firewalls constituyen una de las primeras líneas de defensa en la seguridad informática, actuando como filtros que regulan el tráfico de red entre diferentes zonas de confianza. Su función principal es permitir o bloquear conexiones basadas en reglas definidas, protegiendo así los sistemas y redes de accesos no autorizados, ataques maliciosos y propagación de software dañino. La correcta configuración y gestión de un firewall es fundamental para mantener la integridad, confidencialidad y disponibilidad de los recursos informáticos.

En entornos modernos, los firewalls no solo controlan el tráfico en función de puertos y direcciones IP, sino que también pueden inspeccionar el contenido de los paquetes, detectar patrones de ataque y adaptarse dinámicamente a nuevas amenazas. Sin embargo, una configuración inadecuada puede generar brechas de seguridad o afectar la operatividad de los servicios legítimos. Por ello, es imprescindible comprender en profundidad las herramientas y técnicas para configurar firewalls de manera efectiva.

Este apartado se centra en dos de las soluciones más utilizadas: Windows Defender Firewall, integrado en sistemas Windows, y UFW (Uncomplicated Firewall), una

interfaz simplificada para iptables en Linux. Se explican conceptos clave, comandos y opciones avanzadas, ejemplos prácticos y buenas prácticas para asegurar una protección robusta y eficiente.

6.6.1 Windows Defender Firewall

6.6.1.1 Conceptos básicos

Windows Defender Firewall es el firewall nativo de Windows, diseñado para proteger los equipos mediante el control del tráfico de red entrante y saliente. Funciona en modo stateful, lo que significa que mantiene el estado de las conexiones para permitir respuestas legítimas sin necesidad de reglas explícitas para cada paquete.

Este firewall permite definir reglas basadas en múltiples criterios: puertos, protocolos, direcciones IP, aplicaciones específicas y perfiles de red (dominio, privado, público). La granularidad de estas reglas facilita una protección adaptada a diferentes escenarios y niveles de seguridad.

Además, Windows Defender Firewall se integra con otras herramientas de seguridad de Windows, como el Centro de Seguridad y las políticas de grupo, permitiendo una gestión centralizada en entornos empresariales.

6.6.1.2 Configuración mediante interfaz gráfica

La interfaz gráfica facilita la creación y gestión de reglas para usuarios con distintos niveles de experiencia. Desde el Panel de Control o la Configuración de Windows, se pueden permitir o bloquear aplicaciones, crear reglas personalizadas y configurar perfiles de red.

Esta interfaz es especialmente útil para tareas rápidas o para usuarios que prefieren evitar la línea de comandos, aunque puede resultar limitada para configuraciones complejas o automatización.

6.6.1.3 Configuración mediante PowerShell

PowerShell ofrece un control avanzado y automatizable sobre Windows Defender Firewall, permitiendo crear, modificar y eliminar reglas con precisión.

- **Listar reglas existentes:**

```
Get-NetFirewallRule
```

Este comando muestra todas las reglas configuradas, permitiendo filtrar por nombre, estado o perfil.

- **Crear regla para permitir tráfico TCP en puerto 80 (HTTP):**

New-NetFirewallRule -DisplayName "Permitir HTTP" -Direction Inbound -Protocol TCP -LocalPort 80 -Action Allow

Aquí, se define una regla que permite el tráfico entrante TCP en el puerto 80, comúnmente usado para servidores web.

- **Bloquear tráfico saliente en puerto 25 (SMTP):**

New-NetFirewallRule -DisplayName "Bloquear SMTP Saliente" -Direction Outbound -Protocol TCP -LocalPort 25 -Action Block

Esta regla bloquea el envío de correos salientes por SMTP, útil para prevenir spam o malware que intente enviar correos sin autorización.

- **Eliminar regla por nombre:**

Remove-NetFirewallRule -DisplayName "Permitir HTTP"

Permite eliminar reglas obsoletas o incorrectas para mantener la configuración limpia y segura.

6.6.1.4 Opciones importantes explicadas

- -Direction: Define si la regla aplica a tráfico entrante (Inbound) o saliente (Outbound), permitiendo un control detallado del flujo de datos.
- -Protocol: Especifica el protocolo de red (TCP, UDP, ICMP), fundamental para definir qué tipo de tráfico se regula.
- -LocalPort: Indica el puerto local al que se aplica la regla, clave para proteger servicios específicos.
- -RemoteAddress: Permite restringir la regla a direcciones IP o rangos específicos, aumentando la seguridad.
- -Action: Define la acción a tomar (Allow para permitir o Block para bloquear).
- -Profile: Asocia la regla a perfiles de red (Domain, Private, Public), adaptando la protección según el entorno.

6.6.1.5 Ejemplo avanzado: Permitir RDP solo desde una IP específica

```
New-NetFirewallRule -DisplayName "Permitir RDP desde IP específica" -Direction  
Inbound -Protocol TCP -LocalPort 3389 -RemoteAddress 192.168.1.100 -Action Allow -  
Profile Domain,Private
```

Esta regla permite conexiones al servicio de Escritorio Remoto (RDP) únicamente desde la dirección IP 192.168.1.100 y solo en perfiles de dominio y privado, bloqueando accesos desde otras ubicaciones.

Ventajas:

Limitar el acceso a servicios críticos como RDP a direcciones IP específicas reduce significativamente la superficie de ataque, previniendo intentos de acceso no autorizados y ataques de fuerza bruta.

6.6.2 UFW (Uncomplicated Firewall) en Linux

6.6.2.1 Conceptos básicos

UFW es una interfaz simplificada para iptables, diseñada para facilitar la configuración de firewalls en sistemas Linux. Su objetivo es ofrecer una herramienta accesible para usuarios y administradores que necesitan proteger sus sistemas sin la complejidad de iptables.

UFW permite definir reglas para permitir o denegar tráfico basado en puertos, protocolos, direcciones IP y rangos, con soporte para perfiles y limitación de conexiones.

6.6.2.2 Comandos básicos

- **Habilitar UFW:**

```
sudo ufw enable
```

Activa el firewall con las reglas configuradas.

- **Deshabilitar UFW:**

```
sudo ufw disable
```

Desactiva el firewall temporalmente.

- **Ver estado y reglas activas:**

```
sudo ufw status verbose
```

Muestra las reglas activas y la política por defecto.

- **Permitir tráfico en puerto 22 (SSH):**

```
sudo ufw allow 22/tcp
```

Permite conexiones entrantes TCP en el puerto 22, necesario para acceso remoto seguro.

- **Bloquear tráfico en puerto 23 (Telnet):**

```
sudo ufw deny 23/tcp
```

Bloquea conexiones entrantes en el puerto 23, que es inseguro y obsoleto.

- **Permitir rango de IPs:**

```
sudo ufw allow from 192.168.1.0/24 to any port 80 proto tcp
```

Permite tráfico HTTP solo desde la subred 192.168.1.0/24.

6.6.2.3 Opciones avanzadas explicadas

- **allow / deny:** Define si el tráfico es permitido o bloqueado.
- **from:** Especifica la dirección o rango IP origen para la regla.
- **to any port:** Indica el puerto destino al que se aplica la regla.
- **proto tcp/udp:** Define el protocolo de transporte.
- **limit:** Limita la tasa de conexiones para prevenir ataques de fuerza bruta.

Ejemplo para limitar conexiones SSH:

```
sudo ufw limit ssh
```

Esta regla permite conexiones SSH pero limita la frecuencia para evitar ataques de fuerza bruta.

6.6.2.4 Ejemplo avanzado: Configurar UFW para servidor web seguro

```
sudo ufw default deny incoming
```

```
sudo ufw default allow outgoing
```

```
sudo ufw allow 80/tcp
```

```
sudo ufw allow 443/tcp
```

```
sudo ufw enable
```

Esta configuración bloquea todo el tráfico entrante excepto HTTP y HTTPS, permitiendo solo conexiones salientes, lo que mejora la seguridad del servidor.

Ventajas:

Establecer políticas restrictivas por defecto y permitir solo el tráfico necesario reduce la superficie de ataque y protege el sistema frente a accesos no autorizados.

6.6.3 Buenas prácticas para configuración de firewall

La configuración de un firewall debe ser parte integral de la estrategia de seguridad de cualquier organización. Algunas buenas prácticas incluyen:

- **Política restrictiva por defecto:** Configurar el firewall para bloquear todo el tráfico entrante y permitir solo lo estrictamente necesario.
- **Reglas específicas y limitadas:** Definir reglas que especifiquen puertos, protocolos y direcciones IP concretas para minimizar riesgos.
- **Revisión periódica:** Auditar y actualizar las reglas regularmente para adaptarse a cambios en la infraestructura y amenazas.
- **Documentación:** Mantener un registro detallado de las reglas y cambios para facilitar la gestión y auditoría.
- **Integración con sistemas de detección:** Complementar el firewall con sistemas IDS/IPS para detectar y responder a ataques.
- **Pruebas de penetración:** Validar la efectividad de las reglas mediante pruebas controladas.
- **Alertas y monitoreo:** Configurar notificaciones para eventos sospechosos o bloqueos frecuentes.

6.7 Comprobación del estado de red y puertos

6.7 Introducción

La comprobación del estado de la red y de los puertos abiertos es una tarea fundamental para la administración y seguridad de sistemas informáticos. Conocer qué servicios están activos, qué puertos están escuchando y cómo se comporta la red permite detectar vulnerabilidades, diagnosticar problemas de conectividad y asegurar que solo los servicios autorizados estén accesibles. Esta labor es clave para prevenir accesos no autorizados, ataques de red y para garantizar la correcta operación de aplicaciones y servicios.

En entornos empresariales y domésticos, la monitorización y auditoría de la red deben realizarse de forma periódica y con herramientas adecuadas que proporcionen información detallada y precisa. Además, la interpretación correcta de los resultados y la aplicación de medidas correctivas son esenciales para mantener la integridad y disponibilidad de los sistemas.

Este apartado profundiza en las principales herramientas y técnicas para comprobar el estado de la red y los puertos en sistemas Windows y Linux, explicando comandos, opciones avanzadas, ejemplos prácticos y buenas prácticas para una gestión eficaz y segura.

6.7.1 Herramientas para comprobar puertos abiertos

6.7.1.1 netstat

netstat es una herramienta clásica disponible en la mayoría de sistemas operativos que permite visualizar conexiones de red activas, puertos abiertos, tablas de enrutamiento y estadísticas de red. Su uso es fundamental para identificar qué servicios están escuchando en qué puertos y detectar conexiones establecidas o en espera.

En Windows, Linux y macOS, netstat ofrece opciones para filtrar y mostrar información detallada.

Ejemplo básico en Linux:

netstat -tuln

- -t: Muestra conexiones TCP.
- -u: Muestra conexiones UDP.
- -l: Lista solo puertos en escucha.

- -n: Muestra direcciones y puertos en formato numérico (sin resolver nombres).

Este comando muestra todos los puertos TCP y UDP que están abiertos y escuchando en el sistema, facilitando la identificación de servicios activos.

Ejemplo avanzado en Windows:

```
netstat -ano | findstr :80
```

- -a: Muestra todas las conexiones y puertos en escucha.
- -n: Muestra direcciones y puertos en formato numérico.
- -o: Muestra el identificador del proceso (PID) asociado a cada conexión.

Este comando permite identificar qué proceso está utilizando el puerto 80, comúnmente usado por servidores web.

Ventajas:

netstat es una herramienta rápida y versátil para obtener una visión general del estado de la red y los puertos, esencial para auditorías y diagnósticos iniciales.

6.7.1.2 ss (socket statistics)

ss es una herramienta moderna y más eficiente que netstat en sistemas Linux, diseñada para mostrar información sobre sockets y conexiones de red. Proporciona resultados más rápidos y detallados, siendo especialmente útil en sistemas con gran cantidad de conexiones.

Ejemplo básico:

```
ss -tulw
```

- -t: Muestra conexiones TCP.
- -u: Muestra conexiones UDP.
- -l: Muestra puertos en escucha.
- -w: Muestra información extendida.

Este comando lista todos los puertos TCP y UDP abiertos y escuchando, con detalles adicionales sobre cada socket.

Ejemplo para filtrar por puerto:


```
ss -tuln | grep ':22'
```

Muestra si el puerto 22 (SSH) está abierto y escuchando.

Ventajas:

ss es más rápido y consume menos recursos que netstat, ideal para sistemas con alta carga o para análisis en tiempo real.

6.7.2 Herramientas para escaneo de puertos

6.7.2.1 nmap

nmap es una herramienta avanzada para escaneo y auditoría de redes, capaz de detectar puertos abiertos, servicios activos, sistemas operativos y vulnerabilidades. Es ampliamente utilizada en seguridad informática para realizar análisis profundos y detallados.

Ejemplo básico:

```
nmap 192.168.1.10
```

Escanea los puertos más comunes en la dirección IP especificada.

Opciones importantes:

- -sS: Escaneo SYN (half-open), rápido y sigiloso.
- -p: Especificar puertos o rangos, por ejemplo -p 1-1000.
- -O: Detectar sistema operativo.
- -v: Modo verbose para más detalles.
- -A: Escaneo agresivo que incluye detección de versión y scripts.

Ejemplo avanzado:

```
nmap -sS -p 1-65535 -O -v 192.168.1.10
```

Escanea todos los puertos TCP con detección de sistema operativo y salida detallada.

Ventajas:

nmap es una herramienta poderosa para auditorías de seguridad, permitiendo identificar puntos débiles y servicios no autorizados.

6.7.2.2 netcat (nc)

netcat es una utilidad versátil para leer y escribir datos a través de conexiones de red TCP o UDP. Puede usarse para escanear puertos, transferir archivos o crear conexiones simples.

Ejemplo para escanear puertos:

```
nc -zv 192.168.1.10 1-1000
```

- -z: Modo escaneo sin enviar datos.
- -v: Modo verbose para mostrar resultados.

Este comando verifica qué puertos están abiertos en el rango 1-1000.

Ventajas:

netcat es ligera y flexible, útil para pruebas rápidas y scripts personalizados.

6.7.3 Comprobación de conectividad y diagnóstico

6.7.3.1 ping

ping es la herramienta básica para comprobar la conectividad entre dos hosts enviando paquetes ICMP Echo Request y esperando respuestas. Permite verificar si un equipo está accesible y medir tiempos de respuesta.

Ejemplo:

```
ping google.com
```

Opciones comunes:

- -c: Número de paquetes a enviar (Linux).

```
ping -c 4 google.com
```

- -t: Tiempo de vida (TTL) del paquete (Windows).

Ventajas:

ping es simple y efectiva para diagnósticos iniciales de red.

6.7.3.2 traceroute / tracert

Estas herramientas muestran la ruta que siguen los paquetes hasta un destino, identificando cada salto intermedio. Son útiles para detectar problemas de enrutamiento o latencia.

- En Linux:

```
traceroute google.com
```

- En Windows:

```
tracert google.com
```

Opciones comunes:

- -m: Máximo número de saltos.

```
traceroute -m 20 google.com
```

Ventajas:

Permiten localizar puntos problemáticos en la ruta de red y optimizar el enrutamiento.

6.7.4 Buenas prácticas para comprobación de red y puertos

- Realizar escaneos periódicos para detectar servicios no autorizados.
- Utilizar herramientas adecuadas según el entorno y necesidades.
- Interpretar correctamente los resultados para evitar falsos positivos.
- Integrar la monitorización con sistemas de alerta y respuesta.
- Documentar y auditar los cambios en la configuración de red.
- Mantener actualizadas las herramientas para aprovechar mejoras y correcciones.
- Respetar la legalidad y ética en el uso de herramientas de escaneo.

La comprobación constante y detallada del estado de la red y los puertos es una práctica indispensable para mantener la seguridad y operatividad de los sistemas. Dominar estas herramientas y técnicas permite a los administradores anticipar problemas, responder rápidamente a incidentes y asegurar que la infraestructura tecnológica funcione de manera óptima y segura.

6.8 Monitorización y alertas

6.8 Introducción

La monitorización y generación de alertas son componentes esenciales en la gestión proactiva de infraestructuras informáticas. Permiten supervisar en tiempo real el estado de sistemas, redes y aplicaciones, detectar anomalías o fallos, y responder rápidamente para minimizar impactos. Sin una monitorización adecuada, los problemas pueden pasar desapercibidos hasta que causan interrupciones graves o pérdidas de datos.

La monitorización abarca la recopilación continua de métricas como uso de CPU, memoria, espacio en disco, estado de servicios, tráfico de red y eventos de seguridad. Las alertas se configuran para notificar a los administradores cuando ciertos umbrales o condiciones críticas se cumplen, facilitando una respuesta inmediata.

Este apartado profundiza en las herramientas y técnicas para monitorizar sistemas Windows y Linux, la configuración de alertas efectivas, ejemplos prácticos y buenas prácticas para mantener una infraestructura segura, estable y eficiente.

6.8.1 Herramientas de monitorización en Windows

6.8.1.1 Monitor de rendimiento (Performance Monitor)

Windows incluye el Monitor de rendimiento, una herramienta gráfica que permite crear contadores personalizados para supervisar métricas específicas del sistema y aplicaciones. Se pueden configurar alertas basadas en valores umbral y generar informes detallados.

- **Características principales:**
 - Monitorización en tiempo real y registro histórico.
 - Amplia variedad de contadores (CPU, disco, red, memoria, procesos).
 - Creación de alertas y acciones automáticas.
 - Exportación de datos para análisis externo.

Ejemplo: Configurar un contador para alertar cuando el uso de CPU supere el 80%.

6.8.1.2 PowerShell para monitorización

PowerShell permite obtener métricas y automatizar la monitorización mediante scripts.

- Obtener uso de CPU:

```
Get-Counter '\Processor(_Total)\% Processor Time'
```

- Monitorizar espacio en disco:

```
Get-WmiObject Win32_LogicalDisk | Select-Object DeviceID,FreeSpace,Size
```

- Script para alertar si el espacio libre es menor al 10%:

```
$discos = Get-WmiObject Win32_LogicalDisk -Filter "DriveType=3"
```

```
foreach ($d in $discos) {
```

```
    $libre = $d.FreeSpace
```

```
    $total = $d.Size
```

```
    $porcentaje = ($libre / $total) * 100
```

```
    if ($porcentaje -lt 10) {
```

```
        Write-Output "Alerta: Espacio en disco $($d.DeviceID) menor al 10%"
```

```
    }
```

```
}
```

Ventajas:

La flexibilidad de PowerShell permite integrar monitorización con otras tareas administrativas y generar alertas personalizadas.

6.8.2 Herramientas de monitorización en Linux

6.8.2.1 Nagios

Nagios es una plataforma de monitorización ampliamente utilizada que permite supervisar hosts, servicios y recursos de red. Ofrece alertas configurables, informes y visualización web.

- **Características:**

- Monitorización de disponibilidad y rendimiento.
- Plugins para extensibilidad.
- Alertas vía email, SMS o sistemas externos.
- Escalabilidad para grandes infraestructuras.

6.8.2.2 Zabbix

Zabbix es otra solución completa para monitorización, con soporte para métricas, logs, visualización y alertas avanzadas.

- **Características:**

- Recolección de datos en tiempo real.
- Soporte para SNMP, IPMI, JMX, y más.
- Configuración flexible de triggers y acciones.
- Dashboard personalizable.

6.8.2.3 Comandos básicos para monitorización rápida

- Uso de CPU y memoria con `top` o `htop`.
- Espacio en disco con `df -h`.
- Estado de servicios con `systemctl status servicio`.
- Logs con `tail -f /var/log/syslog`.

6.8.2.4 Ejemplo de script para alertar uso alto de CPU

```
#!/bin/bash
```

```
CPU_USAGE=$(top -bn1 | grep "Cpu(s)" | awk '{print $2 + $4}')
```

```
THRESHOLD=80
```

```
if (( $(echo "$CPU_USAGE > $THRESHOLD" | bc -l) )); then
```

```
echo "Alerta: Uso de CPU alto - $CPU_USAGE%" | mail -s "Alerta CPU"  
admin@dominio.com
```

```
fi
```

Ventajas:

Permite automatizar la detección de condiciones críticas y notificar al equipo de administración para una respuesta rápida.

6.8.3 Configuración de alertas efectivas

Para que las alertas sean útiles, deben estar bien configuradas para evitar falsos positivos y garantizar que los administradores reciban información relevante y accionable.

- Definir umbrales basados en el comportamiento normal del sistema.
- Priorizar alertas según impacto y urgencia.
- Configurar canales de notificación adecuados (email, SMS, sistemas de tickets).
- Implementar escalado de alertas para asegurar respuesta.
- Revisar y ajustar periódicamente las configuraciones.

6.8.4 Buenas prácticas en monitorización y alertas

- Monitorizar tanto recursos físicos (CPU, memoria, disco) como servicios y aplicaciones.
- Integrar monitorización con sistemas de gestión y respuesta a incidentes.
- Documentar configuraciones y procedimientos de respuesta.
- Capacitar al personal para interpretar alertas y actuar adecuadamente.
- Mantener actualizadas las herramientas y plugins de monitorización.
- Realizar pruebas periódicas de las alertas para validar su efectividad.

La monitorización y generación de alertas son pilares para una administración eficiente y segura de infraestructuras tecnológicas. Dominar estas técnicas y herramientas permite anticipar problemas, reducir tiempos de inactividad y mantener la confianza en los sistemas y servicios ofrecidos.

Conclusiones

La seguridad informática y las comunicaciones seguras constituyen pilares fundamentales en el diseño, implementación y mantenimiento de sistemas tecnológicos modernos. A lo largo de esta unidad temática, se ha profundizado en los conceptos, herramientas y técnicas esenciales para proteger la integridad, confidencialidad y disponibilidad de la información en entornos tanto locales como distribuidos. Las conclusiones que se derivan de este estudio reflejan la complejidad y la importancia crítica de estos aspectos en la gestión de infraestructuras informáticas.

En primer lugar, la gestión adecuada de cuentas de usuario y contraseñas seguras es la base para controlar el acceso a los sistemas. La correcta administración de identidades, junto con políticas robustas de contraseñas y autenticación multifactor, reduce significativamente el riesgo de accesos no autorizados y ataques de fuerza bruta. Se ha evidenciado que la implementación de controles de acceso granulares y la educación continua de los usuarios son elementos clave para fortalecer esta primera línea de defensa.

En segundo lugar, la configuración y gestión de permisos de acceso y directivas locales permiten establecer un marco de seguridad que limita las acciones que pueden realizar los usuarios y procesos dentro del sistema. La aplicación de principios como el de menor privilegio y la segregación de funciones contribuyen a minimizar el impacto de posibles brechas de seguridad. Además, la correcta configuración de políticas locales y de grupo facilita la estandarización y el cumplimiento de normativas internas y externas.

El control de procesos y servicios del sistema, otro aspecto central de esta unidad, es vital para garantizar la estabilidad y seguridad operativa. La capacidad para monitorizar, iniciar, detener y configurar servicios y procesos permite mantener un entorno controlado, evitando la ejecución de software malicioso o no autorizado. La comprensión profunda de las herramientas disponibles en sistemas Windows y Linux, así como la automatización mediante scripts, potencia la eficiencia administrativa y la capacidad de respuesta ante incidentes.

Las herramientas de optimización y diagnóstico juegan un papel crucial en la identificación temprana de problemas y en la mejora continua del rendimiento del sistema. La monitorización constante de recursos, la realización de análisis detallados y la aplicación de ajustes basados en datos objetivos permiten anticipar fallos y optimizar el uso de hardware y software. La integración de estas herramientas con sistemas de alerta y respuesta automatizada es una práctica recomendada para mantener la resiliencia de la infraestructura.

En el ámbito de las comunicaciones seguras, la implementación de tecnologías como VPN, HTTPS y SSH es indispensable para proteger la información en tránsito. Estas tecnologías garantizan que los datos no puedan ser interceptados o alterados, y que las partes involucradas en la comunicación sean auténticas. La correcta configuración

y mantenimiento de estas soluciones, junto con la adopción de buenas prácticas, aseguran la confidencialidad y la integridad de las comunicaciones, aspectos críticos en entornos corporativos y de alta sensibilidad.

La configuración de firewalls, tanto en entornos Windows como Linux, es otro componente esencial para la defensa perimetral y la segmentación de redes. La definición de políticas restrictivas, la creación de reglas específicas y la revisión constante de la configuración permiten controlar el tráfico de red y prevenir accesos no autorizados. La combinación de firewalls con otras medidas de seguridad, como sistemas de detección de intrusiones, fortalece la postura defensiva global.

La comprobación del estado de red y puertos, junto con la monitorización y generación de alertas, constituyen prácticas indispensables para la administración proactiva de sistemas. Estas actividades permiten detectar anomalías, responder rápidamente a incidentes y mantener la operatividad de los servicios. La utilización de herramientas especializadas y la configuración adecuada de umbrales y notificaciones son fundamentales para una gestión eficaz.

Finalmente, la integración de todos estos elementos en una estrategia coherente y coordinada de seguridad informática es lo que garantiza la protección efectiva de los activos digitales. La seguridad no es un estado estático, sino un proceso dinámico que requiere actualización constante, formación continua y adaptación a nuevas amenazas y tecnologías.

En conclusión, la unidad temática ha proporcionado un marco teórico y práctico sólido para comprender y aplicar los principios y técnicas de seguridad informática y comunicaciones seguras. La adquisición de estos conocimientos es esencial para cualquier profesional de la informática que aspire a diseñar, implementar y mantener sistemas seguros y confiables en un mundo cada vez más interconectado y vulnerable a ataques cibernéticos.

La seguridad informática y las comunicaciones seguras deben ser consideradas como un proceso integral que abarca desde la gestión de identidades y accesos hasta la protección de las comunicaciones y la monitorización continua de los sistemas. La correcta aplicación de las herramientas y técnicas estudiadas en esta unidad contribuye a crear entornos tecnológicos robustos, capaces de resistir y recuperarse de incidentes de seguridad.

Además, la colaboración entre los diferentes niveles de la organización, la formación constante del personal y la adopción de estándares y normativas internacionales son factores clave para el éxito de cualquier estrategia de seguridad. La tecnología por sí sola no es suficiente; es necesario un enfoque holístico que combine tecnología, procesos y personas.

Por último, es importante destacar que la seguridad informática es un campo en constante evolución. Las amenazas se vuelven cada vez más sofisticadas y los

atacantes emplean técnicas avanzadas para vulnerar sistemas. Por ello, la actualización continua de conocimientos, la implementación de nuevas tecnologías y la revisión periódica de las políticas y configuraciones son imprescindibles para mantener un nivel adecuado de protección.

Esta unidad temática ha sentado las bases para que los profesionales puedan enfrentar estos desafíos con una visión clara, herramientas adecuadas y un enfoque estratégico, contribuyendo así a la construcción de infraestructuras seguras y confiables que soporten el desarrollo y la innovación tecnológica.

En resumen, la seguridad informática y las comunicaciones seguras son elementos inseparables en la gestión moderna de sistemas. La protección de la información y la garantía de la continuidad del negocio dependen en gran medida de la correcta aplicación de los conocimientos y prácticas abordados en esta unidad. La formación y la experiencia adquiridas aquí constituyen un activo valioso para cualquier profesional que desee contribuir a un entorno digital más seguro y resiliente.

¿Qué has aprendido?

En esta unidad he aprendido que la seguridad informática no es un añadido, sino un componente esencial en la administración de sistemas. La gestión adecuada de usuarios y contraseñas es la primera línea de defensa, reforzada por políticas robustas y autenticación multifactor.

He comprendido cómo los permisos y directivas locales permiten establecer marcos de seguridad, aplicando principios de mínimo privilegio y segmentación. También he visto la importancia de monitorizar procesos y servicios, utilizando herramientas tanto en Windows como en Linux para mantener el sistema estable y seguro.

La optimización y el diagnóstico son tareas continuas que permiten anticipar problemas y garantizar un rendimiento eficiente. Además, la adopción de comunicaciones seguras mediante VPN, HTTPS y SSH asegura que la información transmitida se mantenga confidencial e íntegra.

Finalmente, la correcta configuración de firewalls, la comprobación del estado de red y puertos, y la implantación de monitorización y alertas constituyen una estrategia proactiva contra las amenazas. Todo ello demuestra que la seguridad es un proceso integral y dinámico que requiere formación constante, buenas prácticas y compromiso organizativo.

Fuentes de información

- Microsoft. (2024). *Windows Security Documentation*. Microsoft Learn. <https://learn.microsoft.com/es-es/windows/security>
- The Linux Foundation. (2024). *Linux man pages*. <https://linux.die.net/man/>
- OpenVPN. (2024). *OpenVPN Documentation*. <https://openvpn.net/>
- WireGuard. (2024). *WireGuard Documentation*. <https://www.wireguard.com/>
- Apache Software Foundation. (2024). *Apache HTTP Server Documentation*. <https://httpd.apache.org/docs/>
- Red Hat. (2024). *SELinux User Guide*. Red Hat. <https://access.redhat.com>
- Zabbix LLC. (2024). *Zabbix Documentation*. <https://www.zabbix.com/documentation/current/>
- Gordon, A. (2023). *Foundations of Cybersecurity*. Springer.

Glosario

- **Autenticación multifactor (MFA):** Mecanismo que combina varios factores de autenticación para reforzar la seguridad.
- **DAC (Discretionary Access Control):** Control de acceso discrecional.
- **MAC (Mandatory Access Control):** Control de acceso obligatorio.
- **RBAC (Role-Based Access Control):** Control de acceso basado en roles.
- **NTFS:** Sistema de archivos de Windows con permisos avanzados.
- **PAM:** Módulos de autenticación en Linux.
- **Systemd:** Sistema de inicio y gestión de servicios en Linux.
- **Perfmon:** Monitor de rendimiento en Windows.
- **Sysinternals:** Suite avanzada de diagnóstico en Windows.
- **OpenVPN / WireGuard:** Protocolos de VPN para comunicaciones seguras.
- **HTTPS:** Versión segura de HTTP basada en TLS.
- **SSH (Secure Shell):** Protocolo seguro para acceso remoto y transferencia de archivos.

- **UFW:** Interfaz simplificada de iptables en Linux.
- **nmap:** Herramienta de escaneo de puertos y auditoría de red.
- **Nagios / Zabbix:** Plataformas de monitorización.

Actividades de evaluación

Proyecto Práctico: Seguridad informática y comunicaciones seguras (UT6)

Este proyecto práctico está dividido en 4 actividades, cada una valorada con un máximo de 10 puntos. A continuación se presentan los enunciados y rúbricas de evaluación.

Actividad 1: Gestión de cuentas de usuario y contraseñas seguras

Se deberá configurar y administrar cuentas de usuario en sistemas Windows y Linux, aplicando políticas de contraseñas seguras, bloqueo tras intentos fallidos y autenticación multifactor.

Criterio	Excelente (2.5 pts)	Notable (2 pts)	Suficiente (1.5 pts)	No realizado (0 pts)
Creación de cuentas	Cuentas creadas correctamente con todos los tipos requeridos.	Cuentas creadas con pequeños errores.	Cuentas incompletas o con errores importantes.	No se crean cuentas.
Políticas de contraseñas	Políticas robustas aplicadas correctamente.	Políticas aplicadas con errores menores.	Políticas incompletas o poco seguras.	No se aplican políticas.
Bloqueo y MFA	Bloqueo tras intentos fallidos y MFA implementados correctamente.	Implementación parcial o con errores.	Implementación incompleta.	No se implementa.

Auditoría	Auditoría completa y correcta de usuarios y contraseñas.	Auditoría con errores menores.	Auditoría incompleta.	No se realiza auditoría.
------------------	--	--------------------------------	-----------------------	--------------------------

Actividad 2: Permisos de acceso y directivas locales

Se deberá gestionar permisos y directivas locales en sistemas Windows y Linux para controlar el acceso a recursos, aplicando modelos DAC, MAC y RBAC.

Criterio	Excelente (2.5 pts)	Notable (2 pts)	Suficiente (1.5 pts)	No realizado (0 pts)
Configuración permisos Windows	Permisos NTFS y ACL configurados correctamente y aplicados.	Permisos configurados con pequeños errores.	Permisos configurados parcialmente.	No se configuran permisos.
Configuración permisos Linux	Permisos y ACL configurados correctamente y aplicados.	Permisos configurados con pequeños errores.	Permisos configurados parcialmente.	No se configuran permisos.
Aplicación de directivas	Directivas locales aplicadas correctamente.	Directivas aplicadas con errores menores.	Directivas aplicadas parcialmente.	No se aplican directivas.
Comparativa modelos	Comparativa clara y detallada entre modelos DAC, MAC y RBAC.	Comparativa con algunos detalles faltantes.	Comparativa incompleta.	No se realiza comparativa.

Actividad 3: Control de procesos y servicios del sistema

Se deberá monitorizar y gestionar procesos y servicios en Windows y Linux para mantener la estabilidad y seguridad del sistema, incluyendo automatización mediante scripts.

Criterio	Excelente (2.5 pts)	Notable (2 pts)	Suficiente (1.5 pts)	No realizado (0 pts)
Monitorización de procesos	Procesos listados y analizados correctamente.	Procesos listados con pequeños errores.	Procesos listados parcialmente.	No se listan procesos.
Gestión de servicios	Servicios gestionados correctamente.	Servicios gestionados con errores menores.	Servicios gestionados parcialmente.	No se gestionan servicios.
Automatización	Scripts para control automático implementados correctamente.	Scripts implementados con errores menores.	Scripts implementados parcialmente.	No se implementan scripts.
Buenas prácticas	Buenas prácticas aplicadas y documentadas.	Buenas prácticas aplicadas parcialmente.	Buenas prácticas poco aplicadas.	No se aplican buenas prácticas.

Actividad 4: Comunicaciones seguras y configuración de firewall

Se deberá configurar comunicaciones seguras (VPN, HTTPS, SSH) y firewalls en Windows y Linux, aplicando buenas prácticas de seguridad.

Criterio	Excelente (2.5 pts)	Notable (2 pts)	Suficiente (1.5 pts)	No realizado (0 pts)
Configuración VPN	VPN configurada y funcionando correctamente.	VPN configurada con pequeños errores.	VPN configurada parcialmente.	No se configura VPN.
Implementación HTTPS	HTTPS configurado con certificado válido y funcionando.	HTTPS configurado con errores menores.	HTTPS configurado parcialmente.	No se configura HTTPS.
Acceso SSH seguro	SSH configurado con claves y restricciones adecuadas.	SSH configurado con errores menores.	SSH configurado parcialmente.	No se configura SSH.
Reglas de firewall	Firewall configurado con reglas adecuadas y seguras.	Firewall configurado con errores menores.	Firewall configurado parcialmente.	No se configura firewall.

Tabla de relación entre contenidos y criterios de evaluación (CE)

Bloque de contenido	Criterios de Evaluación (CE) asociados
Gestión de usuarios y contraseñas	RA4CEa, RA4CEb, RA6CEa
Permisos y directivas locales	RA4CEc, RA6CEb, RA6CEc
Control de procesos y servicios	RA4CEd, RA6CEb, RA6CEc
Optimización y diagnóstico	RA7CEa, RA7CEc
Comunicaciones seguras (VPN/HTTPS/SSH)	RA6CEd, RA6CEe, RA6CEf
Configuración de firewalls	RA6CEe, RA6CEf
Estado de red y puertos	RA6CEc, RA6CEe
Monitorización y alertas	RA6CEe, RA6CEf, RA7CEc

Rúbrica de evaluación

Indicador de logro	Excelente (10-9)	Notable (8-7)	Aprobado (6-5)	Insuficiente (<5)
RA4CEa/b: Gestiona usuarios y contraseñas	Crea y administra usuarios con políticas seguras y MFA	Crea y administra usuarios con contraseñas robustas	Gestiona usuarios básicos sin políticas completas	No logra gestionar usuarios correctamente
RA4CEc: Aplica permisos y directivas locales	Configura permisos NTFS/Linux y directivas locales avanzadas	Configura permisos en un sistema con alguna limitación	Aplica permisos básicos sin directivas	No gestiona permisos ni directivas
RA4CEd/RA6CEb: Controla procesos y servicios	Supervisa y automatiza servicios en Windows y Linux con scripts	Gestiona servicios con herramientas nativas	Gestiona procesos básicos de forma manual	No controla procesos ni servicios
RA6CEd/e/f: Configura comunicaciones seguras y firewalls	Implementa VPN, HTTPS, SSH y firewalls con buenas prácticas	Configura servicios seguros con alguna limitación	Configura parcialmente protocolos seguros	No logra configurar comunicaciones seguras
RA7CEa/c: Usa herramientas de diagnóstico y documentación	Documenta y aplica herramientas avanzadas (Sysinternals, Nagios, Zabbix)	Aplica herramientas básicas con documentación parcial	Aplica herramientas mínimas sin documentación	No aplica herramientas ni documenta